



# 计算机图形学基础

胡事民

清华大学计算机科学与技术系



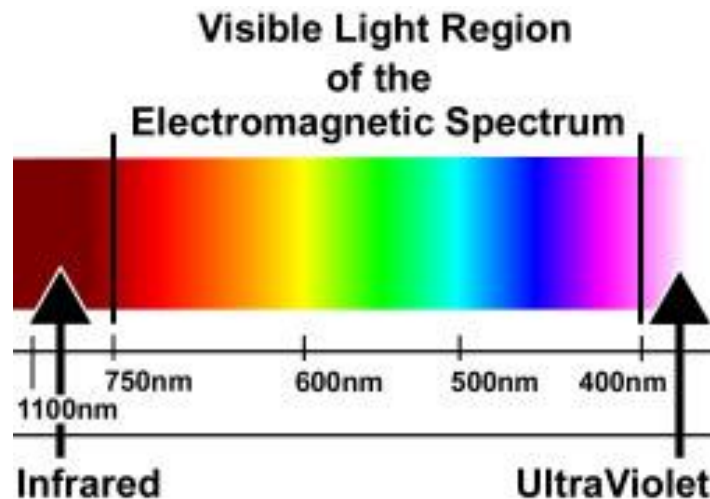
# 本节课程内容

- 图形学中的一些重要概念
  - 颜色视觉
  - 图像和像素
  - 三角网格模型
  - Phong光照模型与明暗处理
  - 视点变换和视点方向
- 课程考核



# 颜色视觉

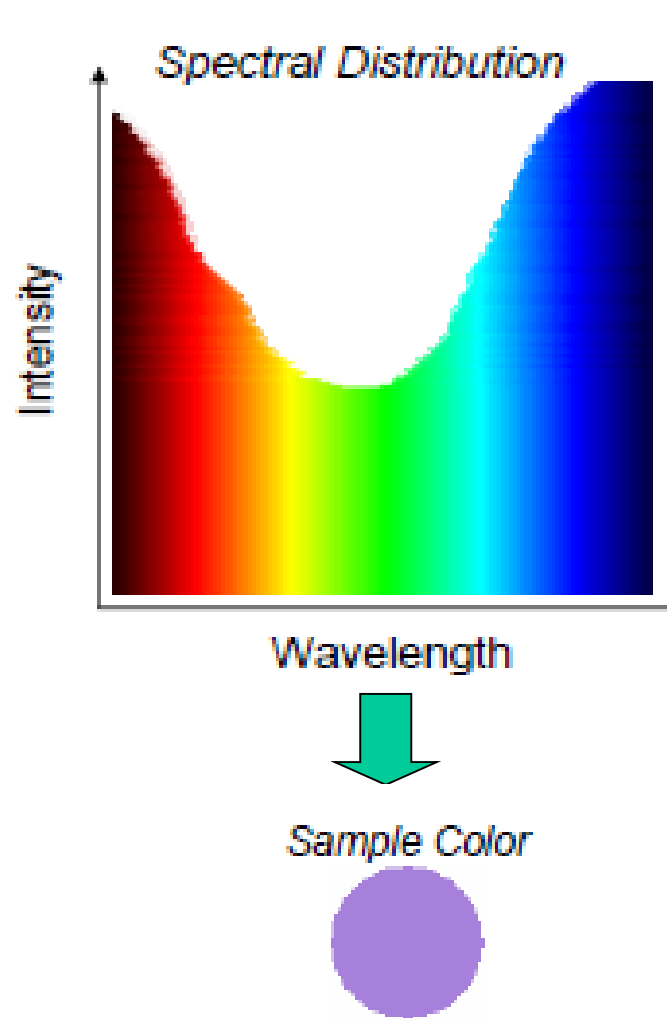
- 什么是颜色?
  - 颜色是人眼对不同波长的光的能量的感知:
    - 不同波长的电磁波 (electromagnetic waves) 对应不同的颜色;
    - 人眼能感知到的光称为可见光, 其波长范围在380nm 到 760nm 之间





# 光的谱分布

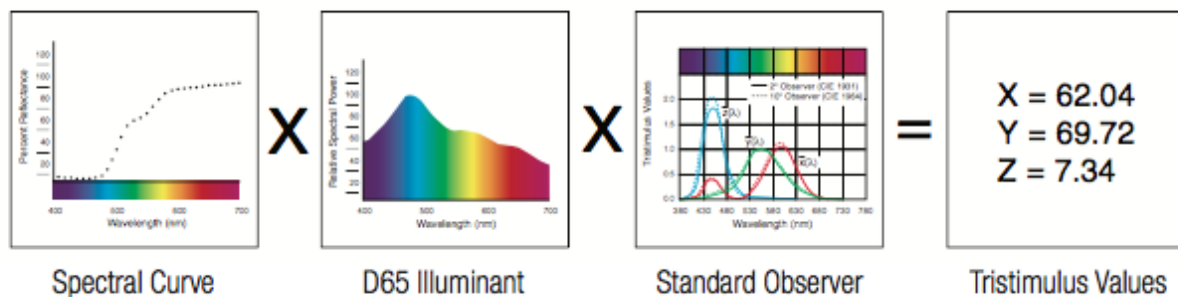
- “光”是由不同波长的电磁波按照某种能量分布混合叠加而成
  - 例如，“白光”是由所有可见波长的电磁波以相等的强度混合得到的
- 谱分布：光在各个可见波长分量的强度分布函数称为光的谱分布





# 光的谱分布

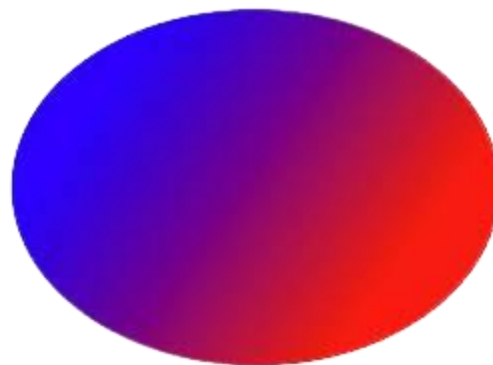
- 与光类似，颜色也可以使用谱分布函数来进行描述
- 人眼感知到的颜色由三个因素决定：
  - 照明条件：光源发出的光的谱分布
  - 物体材质：物体的反射光谱，描述物体对各个波长的光的吸收/反射能力（下节课：BRDF）
  - 观察条件：光对人眼中视觉感受器的刺激

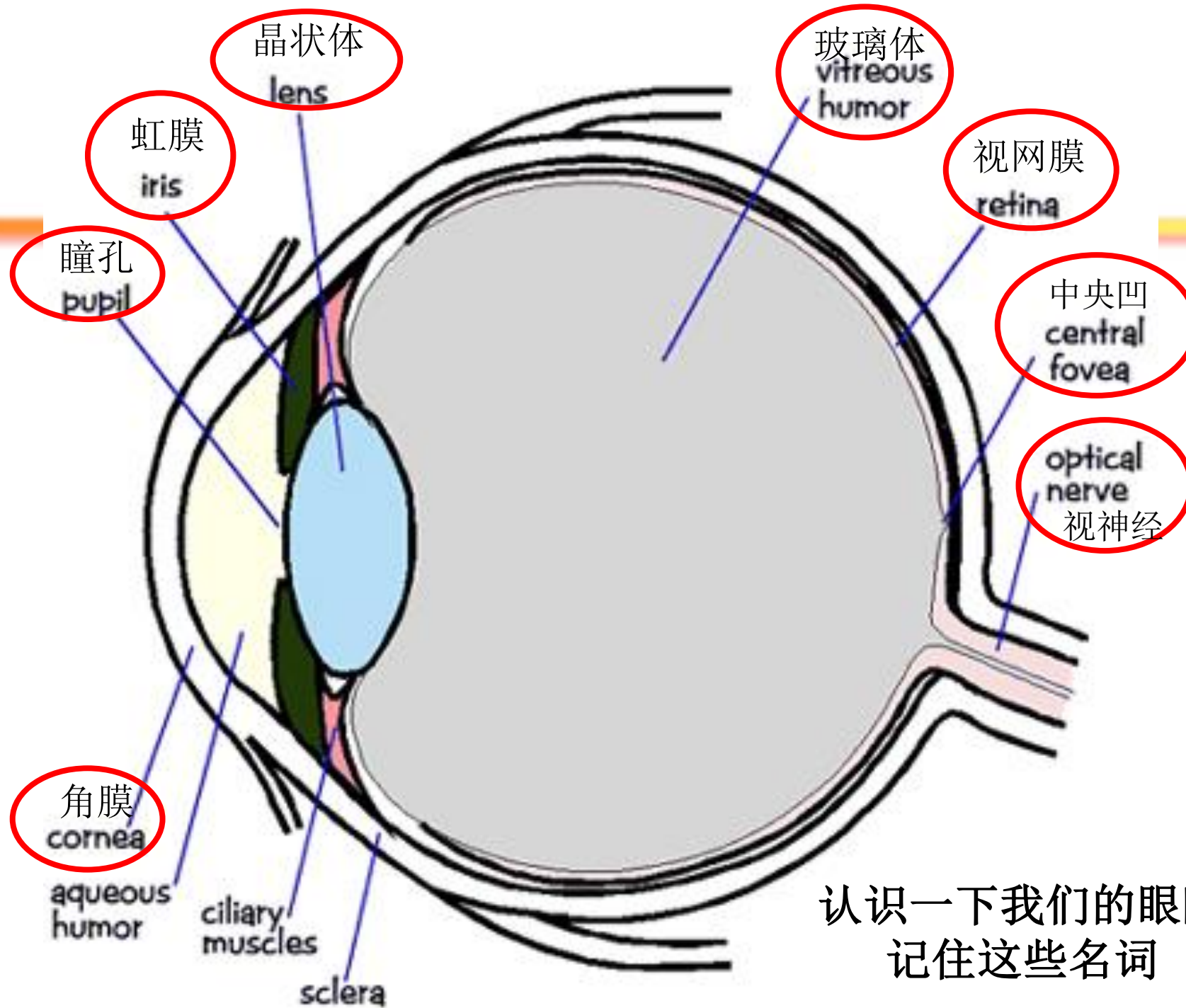




# 异谱同色

- 控制观察条件与物体材质不变，在不同照明条件下，仍然可能被感知为相同的颜色
  - 例：只反射红光的物体，在白光和红光照明下均为红色
- 异谱同色现象：观察条件不变，不同的物体材质仍然可能在某种特定的照明条件下被感知为同一种颜色
  - 例如：紫光（400-450nm）的颜色可以由混合红光（620-750nm）和蓝光（450-495nm）得到。

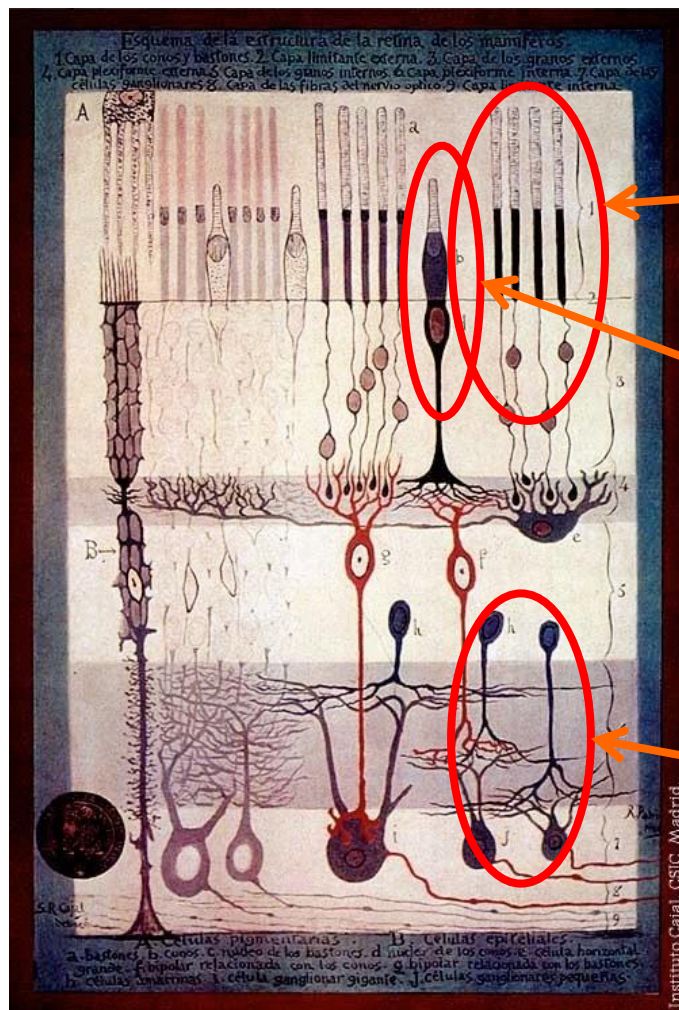




认识一下我们的眼睛  
记住这些名词



# 视网膜——我们的感知器!



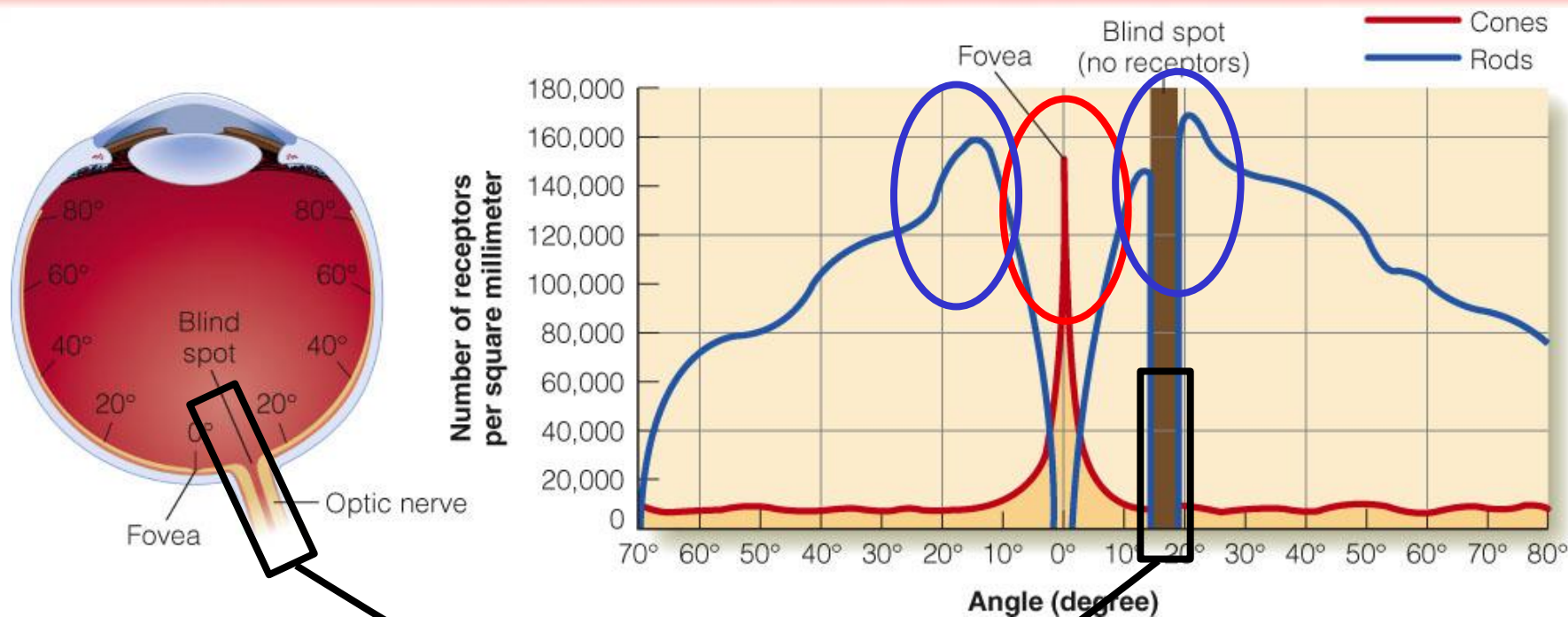
**杆状细胞(Rod),**  
对亮度更敏感（因而在夜间的  
视力由杆状细胞主导）

**锥状细胞(Cone),**  
感知色彩，主要分为对红光敏  
感，对绿光敏感，对蓝光敏感的  
三类。

**神经节细胞(Ganglion),**  
神经节细胞负责将这些细胞  
受到的刺激传递给视神经。



# 杆状细胞与锥状细胞的密度



右图显示了各种感光细胞在视网膜上的分布。横坐标是相对于中央凹的角度，纵坐标则是相应的感光细胞数目。其中的红色曲线是锥细胞在视网膜上的分布，我们可以看出锥细胞主要集中在中央凹处。因而人眼在视野中心处辨色力最强。



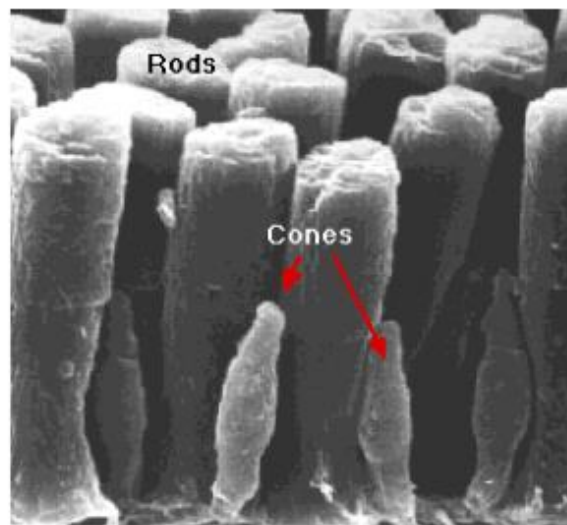
# 颜色空间

- 以数学的方式描述、分类和重现颜色
- 常用的颜色空间有：
  - RGB: 电子显示设备
  - CMY: 印刷
  - HSV: 更直观, 便于调整
  - CIE XYZ: 科学研究和标准化

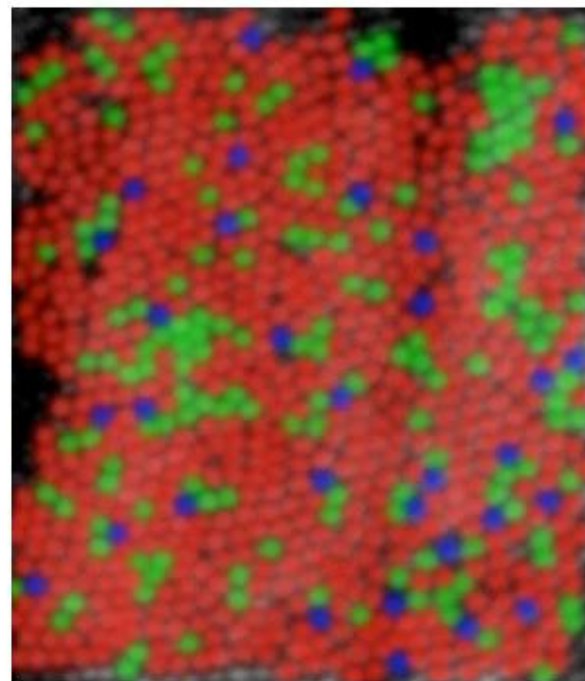


# 为何采用RGB 颜色空间？

- **模拟人类视觉：**左图是视网膜组织的显微镜成像图。人眼中大约有1.2亿个杆状细胞与五六百万个锥状细胞。右图中细胞显示的颜色表示它哪种颜色的光刺激最敏感。



120 million rods  
5-6 million cones



R 63% - G 31% - B 6%



# RGB 颜色空间

- 在所有用于表示色彩的颜色空间中，**RGB (红绿蓝)** 颜色空间在计算机图形学中的使用最为广泛：
  - 一种颜色通过三通道向量  $(r, g, b)$  来表示；
  - 部分常用操作可以转化为对 RGB 三通道分别独立处理；
  - 在计算机中，通常将  $r, g, b$  三个分量分别规整化为  $[0, 1]$  内的浮点数；或  $[0, 255]$  内的8bit无符号整数



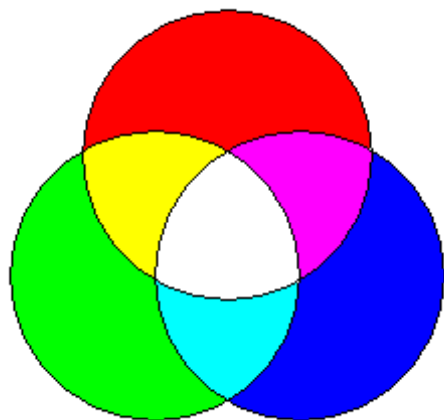
# RGB 颜色空间

- 具体来说，颜色表示为三个基本色：**红色 (R)**，**绿色 (G)**，**蓝色 (B)** 的线性组合：

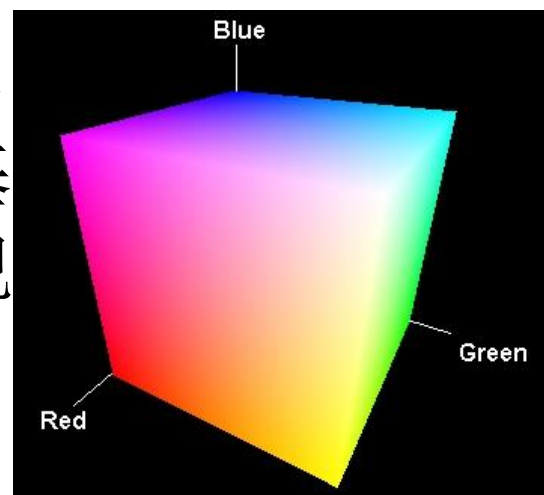
- $C = rR + gG + bB$

- 为什么

- 以人
    - 人眼对红、绿、蓝三



基本颜色  
数理论为基  
三种锥状视



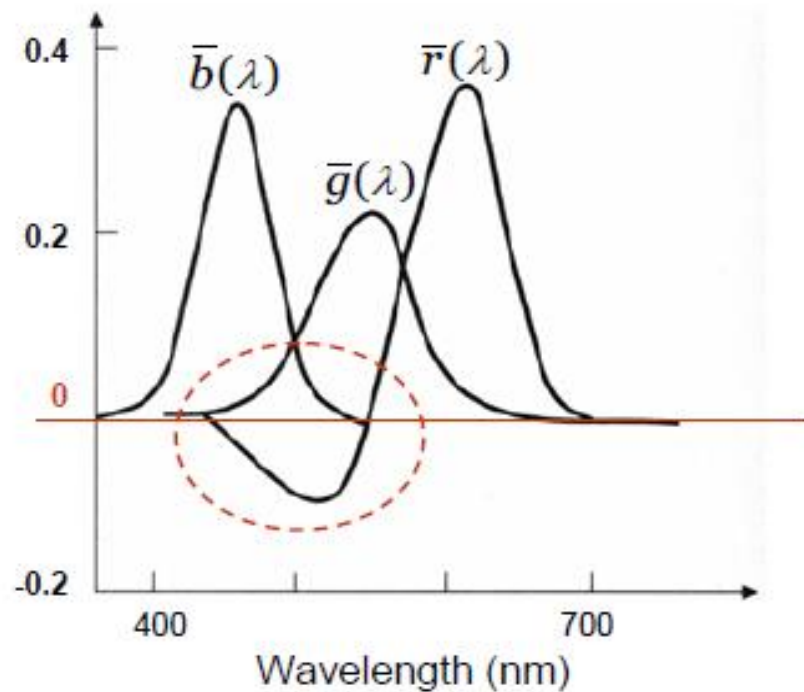
别对红、绿、

*RGB 颜色空间的图示*



# RGB 颜色空间

- 然而，一部分色彩无法表示成 R, G, B 光波的 **正线性组合**，这是 RGB 颜色空间的一个缺点
- 右图为所有可见光对应的 RGB 分量图：





# CMY 颜色空间

- CMY: 采用不同于 RGB 的另外一组基本颜色:
  - Cyan (青), Magenta (品红), Yellow (黄): 分别是 R, G, B 的补色 (complements)

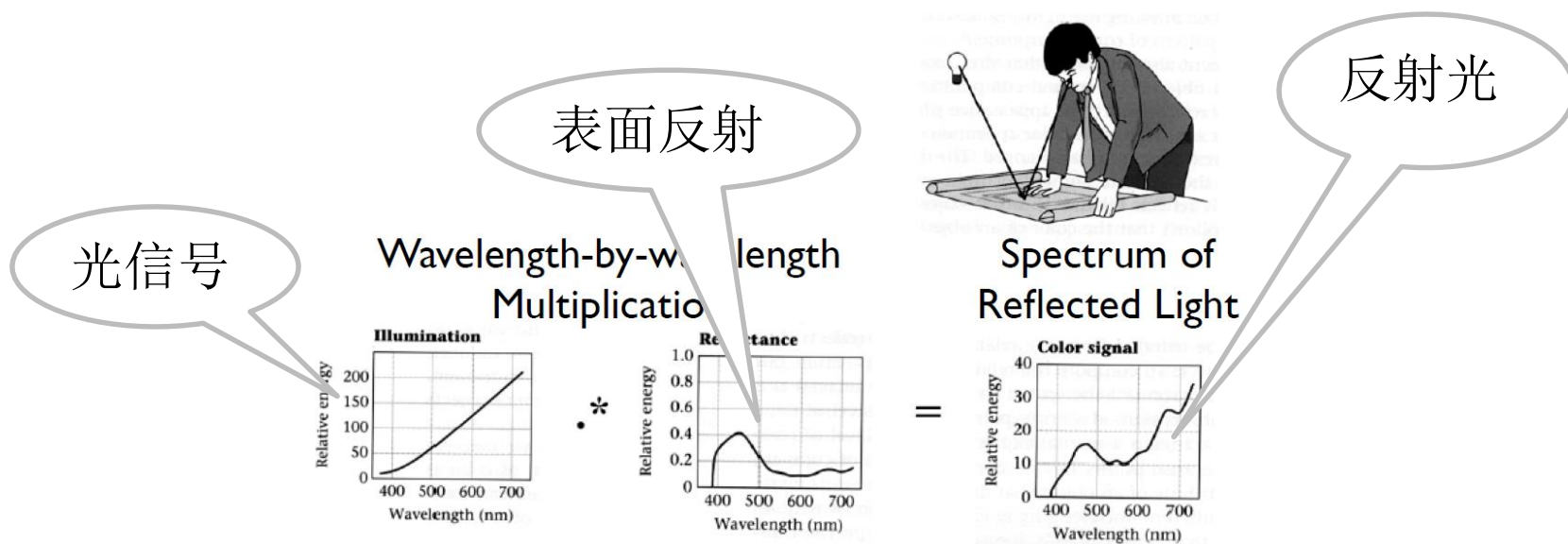
$$\begin{array}{c} \text{C} \\ \text{M} \\ \text{Y} \end{array} = \begin{array}{c} 1 \\ 1 \\ 1 \end{array} - \begin{array}{c} \text{R} \\ \text{G} \\ \text{B} \end{array}$$

- CMY 被称为“减色系统”，而RGB 被称为“加色系统”：
  - RGB — (0, 0, 0)为黑，(1, 1, 1)为白
  - CMY — (1, 1, 1)为黑，(0, 0, 0)为白，  
印刷业只能用CMY模型



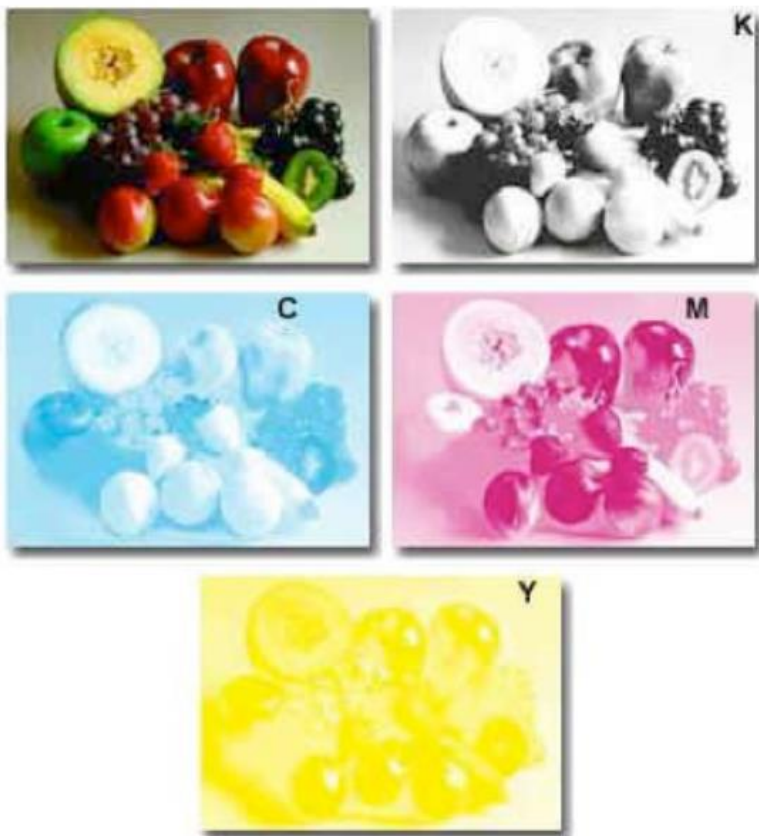
# RGB（加色）-CMY（减色）

- 加性混合主要应用于主动发光的物体，如液晶显示器、电视机等。
- 减性混合主要应用于被动发光的物体，其颜色由物体表面的反射参数决定。





- 打印机的CMYK颜色系统（减性混合）

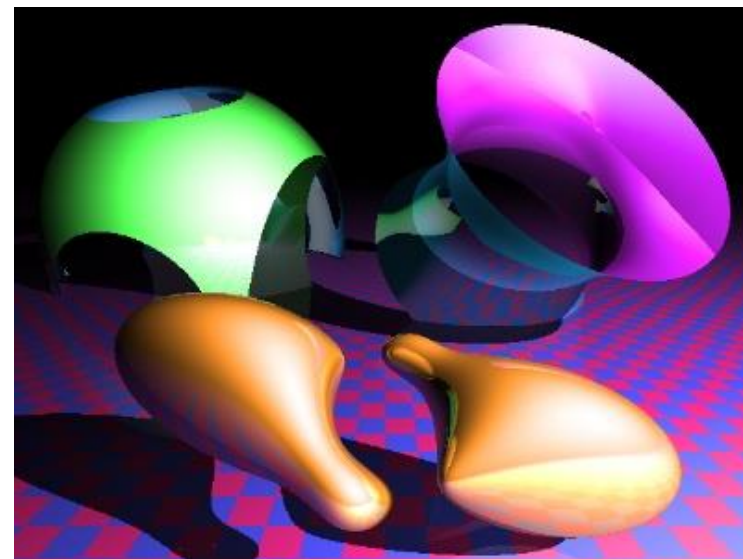


1. 油墨本身并不能发光，打印出物体的颜色是通过反射光的形式表现出来的。
2. 仅用CMY三种颜色的最深的颜色是深灰色，很难达到纯黑，所以添加黑色(Key Color)来拓宽颜色表示。



# HSV 颜色空间

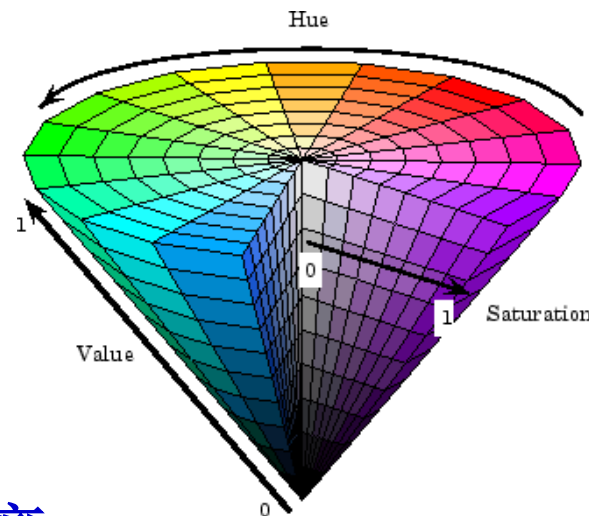
- 在8bit的 RGB 颜色空间中，可以包含多达 $256*256*256 =$  **16,777,216** 种可能的颜色
- RGB 颜色空间不够符合直觉，无法做出更鲜艳/更亮之类的调整，难以描述和定位如此大量的颜色
- HSV 系统则提供了一个直观的方法来对颜色进行准确的选择
- HSV 颜色空间应用于：艺术领域，以及图像处理、分形图像...





# HSV 颜色空间

- HSV：圆锥形的颜色空间
  - **Hue (色调)** 也叫色相，它描述了色彩的本征属性，即我们常说的一个色彩是红、橙、黄、绿、青、蓝、紫，等等
  - **Saturation (饱和度)** 也叫纯度：  
饱和度越低，色彩越白
  - **Value of brightness (亮度)**：  
亮度越低，色彩越黑



HSV更好地描述了人眼对颜色的感知距离，  
大部分软件采用基于HSV空间的调色板



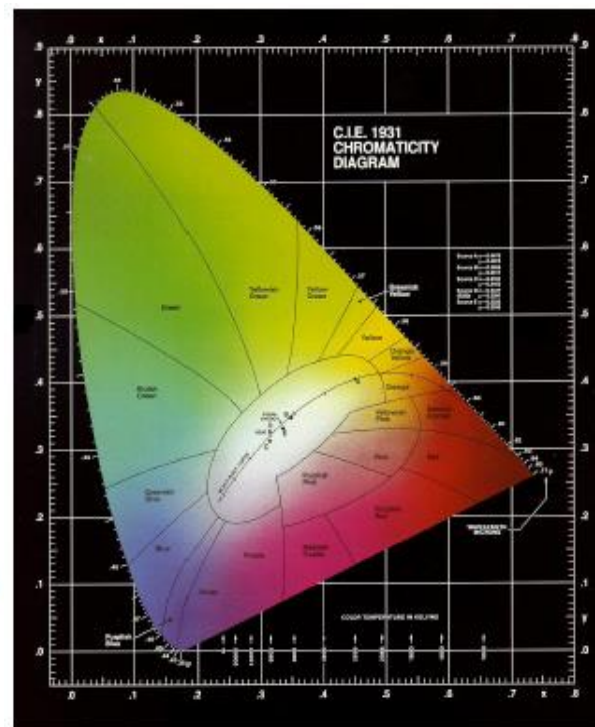
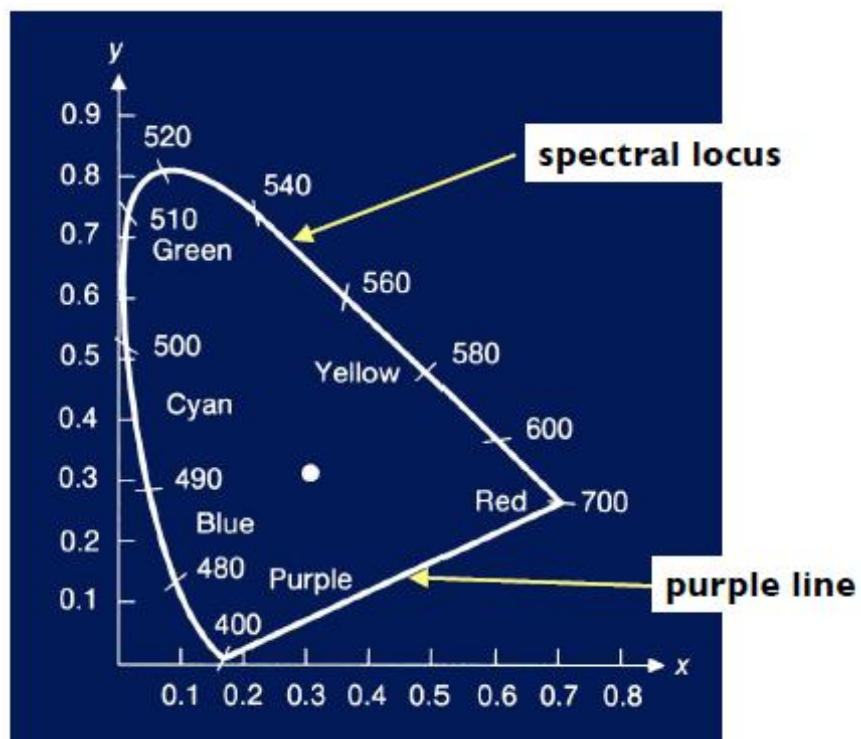
# CIE XYZ 颜色空间

- CIE XYZ 颜色空间
  - 由国际发光照明委员会于1931年提出
- CIE XYZ 色彩空间可以表示所有可感知的颜色 ( RGB 空间不能)
  - 颜色基 XYZ 是 颜色基 RGB 的线性变换
  - 更多地应用于颜色科学的研究
  - 与人类的感知系统密切相关



# CIE XYZ 色度图

- 可以将 CIE XYZ 颜色空间可视化为如下色度图，其中： $x = X/(X+Y+Z)$ ； $y = Y/(X+Y+Z)$





比较直观，适合于艺术领域的颜色模型是哪种？

- ☐ A RGB模型
- ☐ B XYZ模型
- ☒ C HSV模型
- ☐ D CMY模型

提交



# 本节课程内容

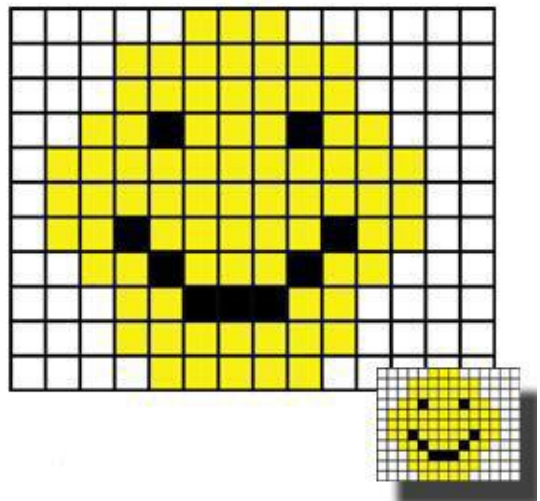
- 图形学中的一些重要概念
  - 颜色视觉
  - 图像和像素
  - 三角网格模型
  - Phong光照模型与明暗处理
  - 视点变换和视点方向
- 课程考核



# 图像和像素

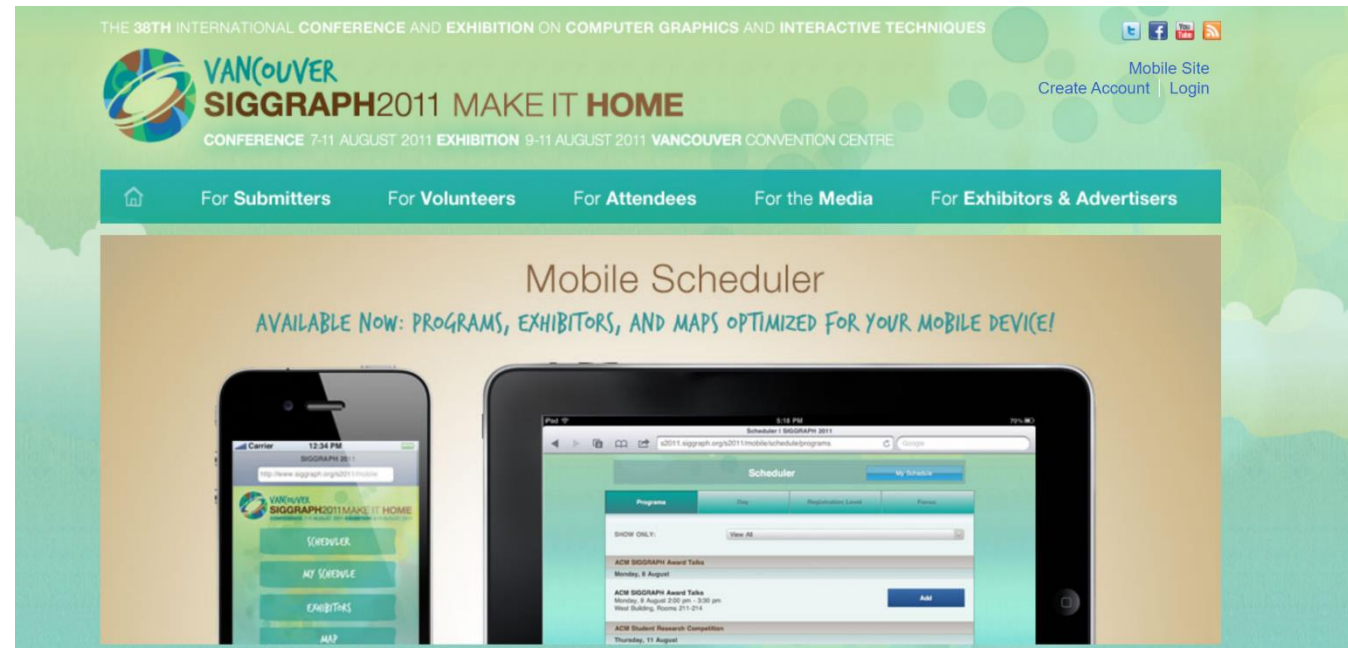
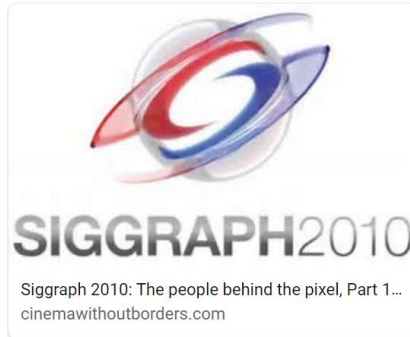
- 图像

- 图像可以看成是一个二维离散函数： $f(x, y)$
- 函数  $f$  的定义域是以矩阵形式排列着的格子，这些格子被称为 **像素 (pixel)**。
- 分辨率：图像中像素的数量（例： $1600 \times 1280$ ）
- 函数  $f$  的值域为各个像素的色彩空间
  - 一维（灰度图）
  - 三维（RGB）
  - 四维（RGBA，其中A表示透明度）
  - .....





# People behind Pixel





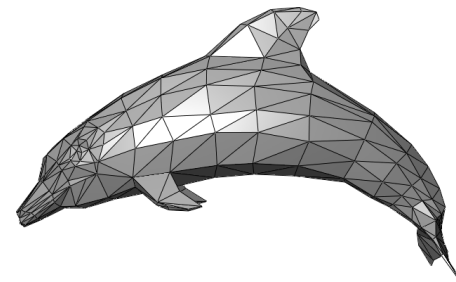
# 本节课程内容

- 图形学中的一些重要概念
  - 颜色视觉
  - 图像和像素
  - 三角网格模型
  - 光照模型与明暗处理
  - 视点变换和视点方向
- 课程考核



# 三角网格模型

- 图形学的基本目标是什么？
  - 从虚拟的三维场景及特定的相机参数中，生成二维图像
  - 三维场景又以怎样的数据结构来表示？
    - 简单球体、长方体可直接用其几何方程描述；对于复杂模型，则需要使用参数曲线和曲面或者更一般的网格(mesh)模型来进行描述
    - 网格模型之中又以三角网格最为常用





# 三角网格的定义

- 三角网格是由一系列欧氏空间中的三维顶点以及连接这些顶点的若干三角面片组成，具体包括：
  - 顶点集合  $V = (v_1, v_2, \dots, v_n)$
  - 面片集合  $F = (f_1, f_2, \dots, f_m)$ 
    - 其中  $F$  中的每个面片  $f_i$  都是由  $V$  中的顶点构成的空间三角形：
$$f_1 = (v_{a1}, v_{b1}, v_{c1}), f_2 = (v_{a2}, v_{b2}, v_{c2}), \dots$$



# 几个例子

- 模型“牛”上显示了三角网格的结构
- 模型“龙”和“人头”是利用三角网格（Smooth Shading模式）进行绘制的结果
- Shading（着色）的概念后面会进行介绍





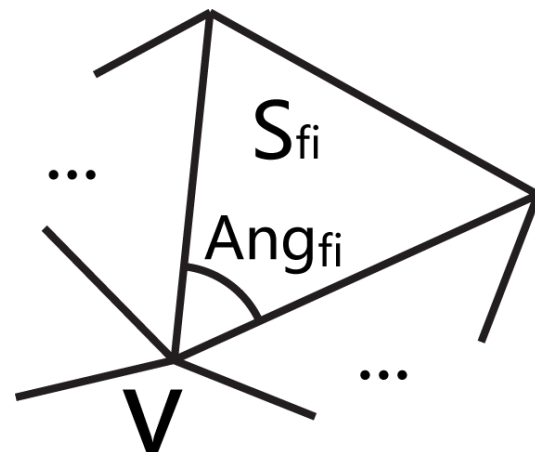
# 法向量

- 三角面片的法向量 (normal):
  - 垂直于该三角面片所在平面的非零向量;
  - 对于每个三角面片而言, 其法向量都有两种可能的朝向;
  - 法向量的朝向决定了三角面片的正面与反面;
  - 对于连续可定向的三角网格整体而言, 相邻的三角面片需要具备一致的法向量朝向。



# 法向量

- 三角网格顶点的法向量可以通过其周围所有三角面片的法向量不同方式的加权平均计算：
  - 假设  $v$  是  $k$  个三角面片  $f_1, f_2, \dots, f_k$  所共用的顶点；
  - 按算数平均计算：
    - $N_v = (N_{f1} + \dots + N_{fk}) / k$
  - 按面积加权平均计算：
    - $N_v = (S_{f1}N_{f1} + \dots + S_{fk}N_{fk}) / (S_{f1} + \dots + S_{fk})$
  - 按角度加权平均计算：
    - $N_v = (Ang_{f1}N_{f1} + \dots + Ang_{fk}N_{fk}) / (Ang_{f1} + \dots + Ang_{fk})$





# 三角网格的简单绘制

- 基于颜色的绘制

- 给三角网格模型的每个顶点指定颜色属性
- 模型表面每个点的颜色通过其所在三角面片顶点的颜色插值得到

- 基于光照的绘制

- 给网格模型每个面片指定正反面的材质属性
- 指定一个虚拟的光照环境
- 如何计算材质在不同光照条件下呈现出来的颜色就涉及到了光照模型的问题



# 本节课程内容

- 图形学中的一些重要概念
  - 颜色视觉
  - 图像和像素
  - 三角网格模型
  - Phong光照模型与明暗处理
  - 视点变换和视点方向
- 课程考核



# 着色 (Shading)

- 为什么需要光照模型？
  - 绘制一张图像，其实就是计算每个像素的颜色值（通常为RGB）
- 着色 (Shading)：通过模拟光照，计算像素对应的颜色
  - 使用光照模型进行着色
- 着色点 (Shading Point)：需要着色的像素对应的场景点
- 着色器 (Shader)：运行在图形处理器 (GPU) 上的完成着色计算的程序
  - 现代图形API (OpenGL、DirectX等) 都支持高度可编程的着色器



# 光照模型

- 光照模型 (lighting model 或 illumination model) 用于计算光的强度：
  - 局部光照 (Local Lighting)模型
    - 关注物体直接受到光源影响所产生的光照效果，即直接光照 (Direct Illumination)
  - 全局光照 (Global Lighting)模型
    - 除了关注直接光源照射的影响，即直接光照
    - 还关注光线在场景中其他物体表面发生折射、反射等作用后，间接照射到物体表面上产生的影响，即间接光照(Indirect Illumination)
    - 真实感更强
    - 全局光照 (Global Illumination) = 直接光照 + 间接光照



# 光照模型的历史

- 1967年, Wylie 等人第一次在显示物体时加入了光照明效果, 认为光的强度与物体到光源的距离成反比关系
- 光照模型关注的问题:
  - 定量地描述物体对（接收到的）光线的反射行为
- 1970年, Bouknight 提出了第一个光反射模型:
  - Lambert 漫反射光 + 环境光
  - 发表于 [Communication of ACM](#)
- 1971年, Gouraud 提出了漫反射模型加插值的思想:
  - Lambert 漫反射光 + Barycentric 插值
  - 发表于 [IEEE transactions on Computers](#)



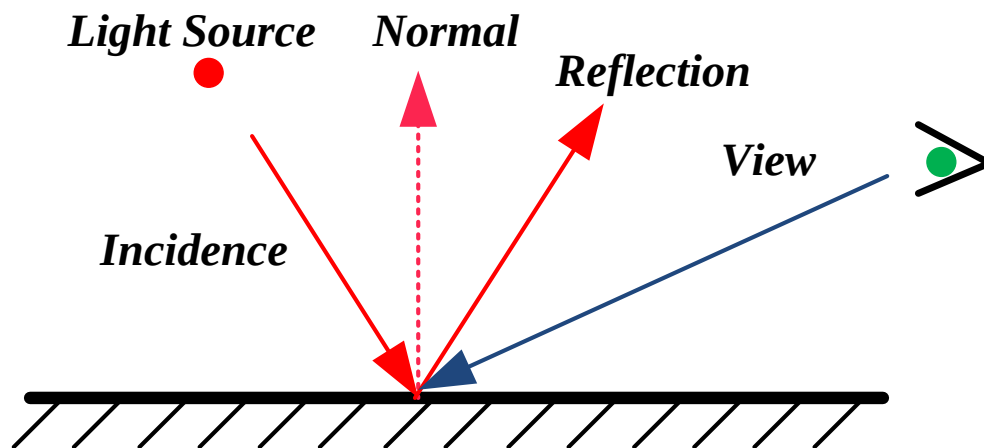
# 光照模型的历史

- 1975年, Phong 提出了图形学中第一个有影响也是最有影响的光照模型: Phong 模型
  - 漫反射 (diffuse light) + 环境光 (ambient light) + 高光 (specular light)
  - 发表于 [Communication of ACM](#)
- 以上光照模型均为[经验模型](#), 不完全符合物理规律
  - 另一方面, 也没有考虑阴影
- 1986年, Jim Kajiya提出渲染方程
- 此后基于物理的渲染 (Physically-Based Rendering, PBR) 逐渐发展与完善
  - 下节课: 辐射度量学与BRDF (双向反射分布函数)



# 光的传播

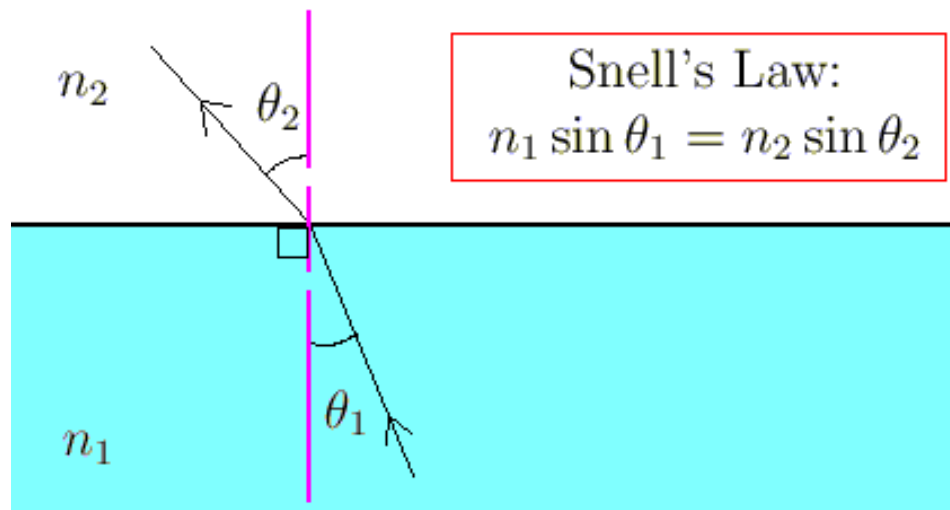
- 光的镜面反射遵循反射定律：
  - 入射角等于反射角；
  - 入射光线、反射光线、以及反射面的法向量位于同一平面内





# 光的传播

- 折射定律 (也叫 Snell 定律):
  - 入射角和折射角的正弦值之比是一个仅仅取决于介质的常数;
  - 这个常数称为相对折射系数



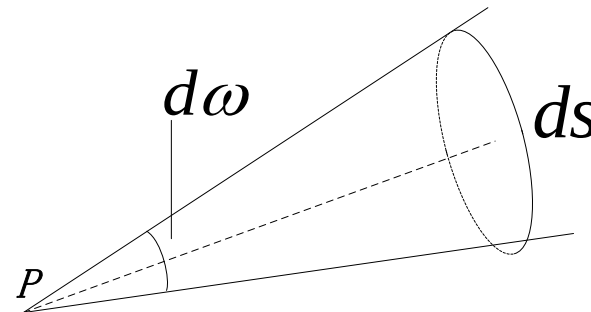


# 光的度量方法

- 立体角 (Solid Angle):

- 衡量物体相对于某一视点 P 的视角大小
- 最大立体角为?  $4\pi$

$$d\omega = \frac{ds}{r^2}$$



- 光通量 (Luminous Flux):

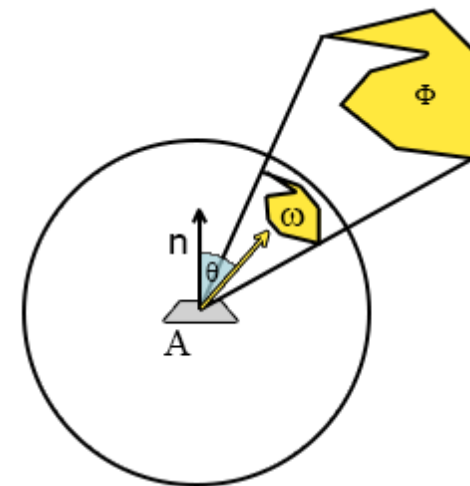
- 单位时间内通过面元 dS 的光能量，记为 dF

- 发光强度 (Intensity):

- 单位立体角内的光通量，记为 I

- 光亮度 (Radiance) :

- 单位立体角单位投影面积内的光通量，记为 L
- 用来描述一根光线携带的能量





# Phong 光照模型

- 问题转化为：物体从发光强度为 $I$ 的光源处接收到能量为 $I(r)$ 的光线，反射出去的光线能量（Radiance, 记为 $L$ ）是多少？
- Phong 模型支持点光源和方向光源
- Phong 模型是局部光照模型，将出射光的能量分解为三个部分：

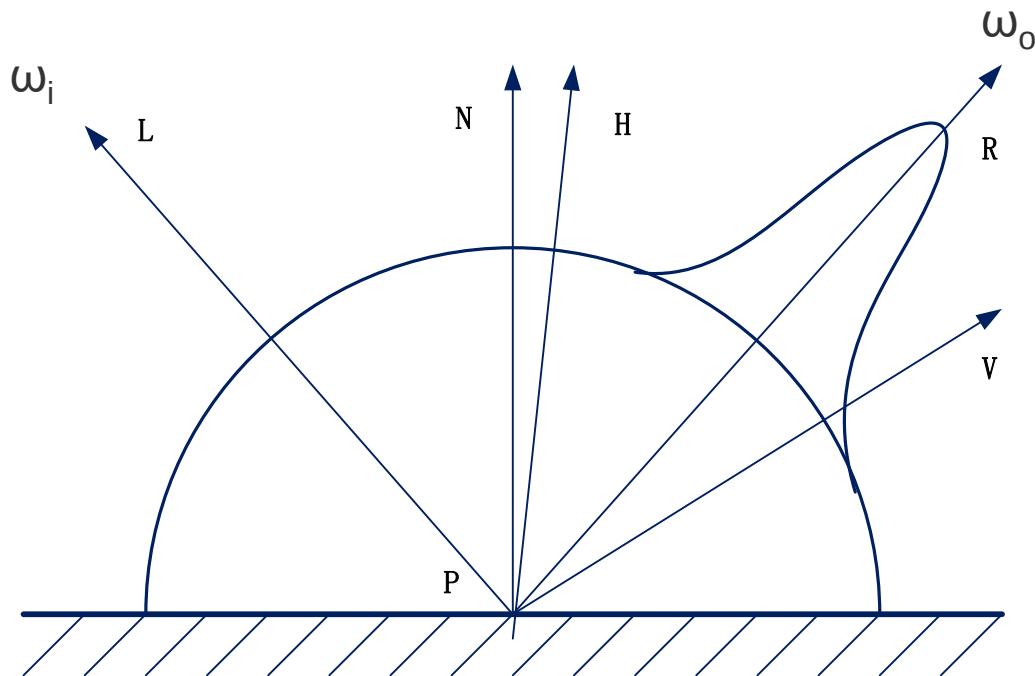
$$L_o = L_d + L_s + L_a$$

- 漫反射（diffuse）
- 镜面反射（specular）
- 环境光（ambient）



# Phong 光照模型

- $\omega_i$  : 入射光方向
- $\omega_o$ : 反射光方向
- $N$ : 着色点 $P$ 的单位法向量
- $V$ : 视点方向
- $H$ : 半程向量, 即 $\omega_i$ 和  $V$  夹角的角平分线方向
  - 作用:  $N$ 与 $H$ 的夹角取代 $\omega_o$ 与 $V$ 的夹角!
  - 在计算镜面反射项时能减少计算量
- 入射光携带的能量用 $L_i(\omega_i)$ 表示





# Phong 光照模型

- 漫反射光 (Diffuse Light) :

- 漫反射光的传播是各向同性的, 与观测方向V没有任何关系;
- 漫反射光的强度为:

$$L_d = K_d L_i(w_i) \max(0, N \cdot w_i)$$

- $K_d$  是漫反射系数, 表征着色点本身对能量吸收的强度
- $K_d$  具有三个分量  $k_{dr}, k_{dg}, k_{db}$  分别代表 R, G, B 三个通道的漫反射系数
- $K_d$  与模型自身的色彩紧密相关
- $N \cdot w_i$  为单位法向量与入射方向的点乘, 即为两者的余弦值, 负数无意义



# Phong 光照模型

- 镜面反射光（Specular Light）：高光效果

- 对于光滑的平面，依据反射定律，反射光线往往集中在一个小的立体角内，这些反射光我们称之为镜面反射光；
- 镜面反射光的强度为：

$$L_s = K_s L_i(w_i) \max(0, w_o \cdot V)^n$$

- $K_s$  是镜面反射系数，与物体表面光滑程度相关；
- 计算时， $w_o \cdot V$  可以用  $N \cdot H$  代替以简化运算（**Blinn-Phong 模型**）
- $n$  是反射指数； $n$  越大，则高光区域越集中



# Phong 光照模型

- 环境光（Ambient Light）：估计间接光照

- 环境光的强度为：

$$L_a = K_a I_a$$

- $K_a$  是物体对环境光的反射系数

- 假设环境光的强度是一个常数  $I_a$ ；但是哪怕对于同一个物体的不同部分，接收到的间接光也并不相同！

- Phong模型作为经验模型，并不完全符合物理规律



# Phong 光照模型

- Phong模型把物体反射出的光线能量（颜色）拆分为三个部分，环境光、漫反射光以及镜面光：

$$\begin{aligned} L_o &= L_d + L_s + L_a \\ &= K_d L_i(w_i) \max(0, N \cdot w_i) + K_s L_i(w_i) \max(0, w_o \cdot V)^n + K_a I_a \end{aligned}$$

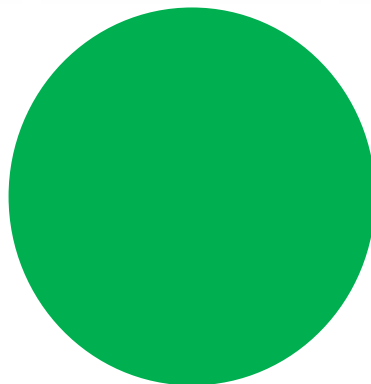


# Phong 模型示例



漫反射光

+



环境光

+



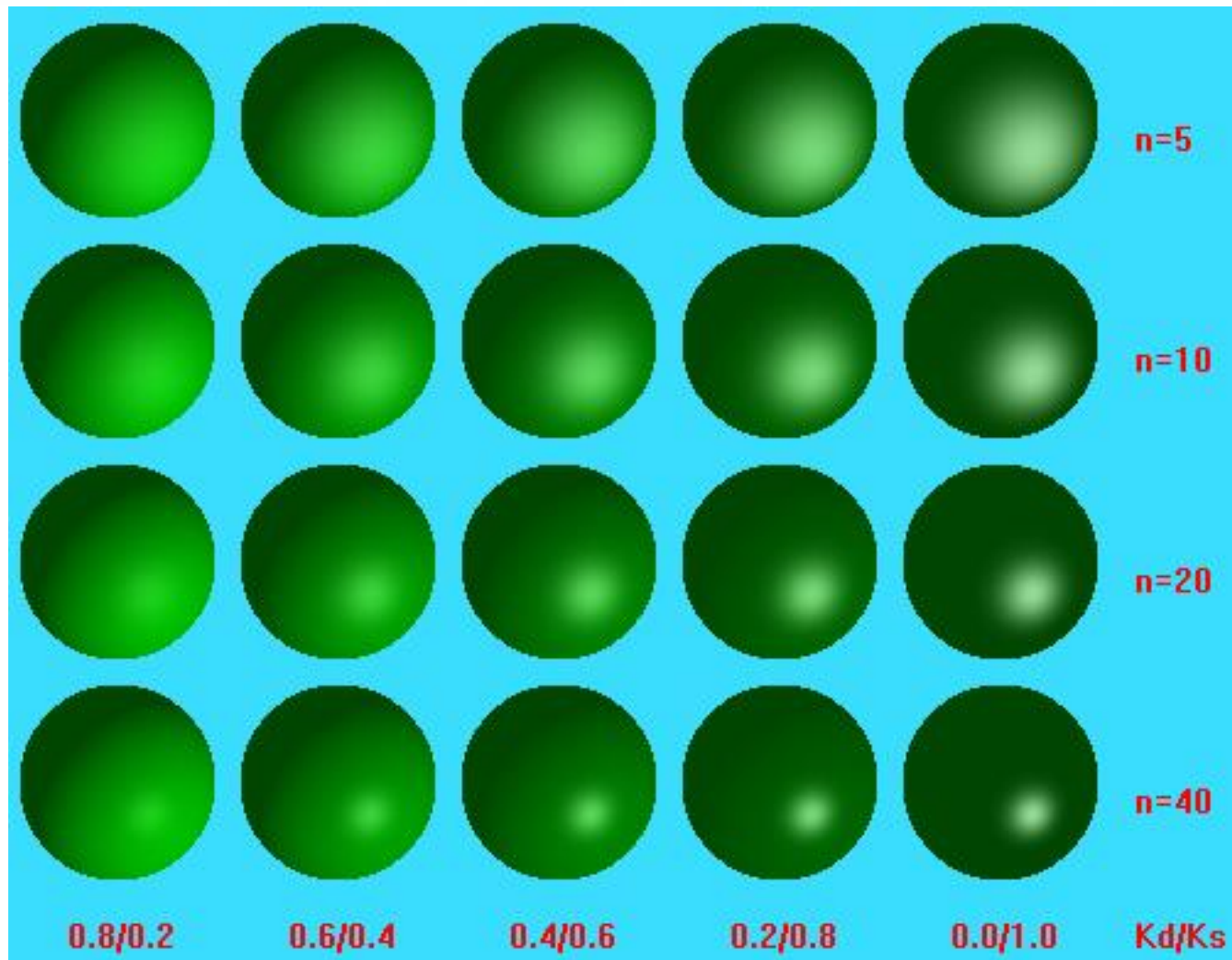
镜面反射光

=





# Phong 模型示例



# Wu Jianhua's Demo

---



Demo



# 明暗处理 (Shading)

- 物体在渲染过程中往往被离散化为三角网格储存
  - 仅仅对物体顶点进行着色：不光滑
  - 通常的明暗处理除了使用光照模型外，还需要进行插值
- Gouraud 明暗处理和 Phong 明暗处理
  - Gouraud 明暗处理是对颜色进行插值
  - Phong 明暗处理则是对法向进行插值
- 为了概念的统一，我们将不做明暗处理的绘制模式称为Flat Shading



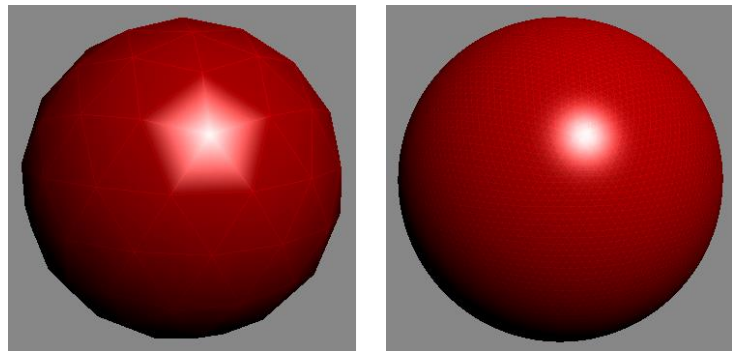
# Gouraud 明暗处理

- 由 Gouraud 于 1971 年提出
  - 又被称为 Gouraud 插值
- 计算方法：
  - 首先计算所有模型顶点的颜色值；
  - 模型上的任何一点的颜色为：其所在三角面片三个顶点的颜色值重心插值 (Barycentric Interpolation) 的结果



# Gouraud 明暗处理

- Gouraud明暗处理的问题：
  - 当三角面片在屏幕空间投影面积较大时，中心区域距离顶点较远，此时只通过顶点颜色的插值难以很好刻画内部区域应有的颜色
  - 这一问题在局部高光区域表现的尤其明显：两个球体模型在相同的光照环境下以相同的速度旋转，当网格比较稀疏的时候，高光区域的过渡极其不平滑





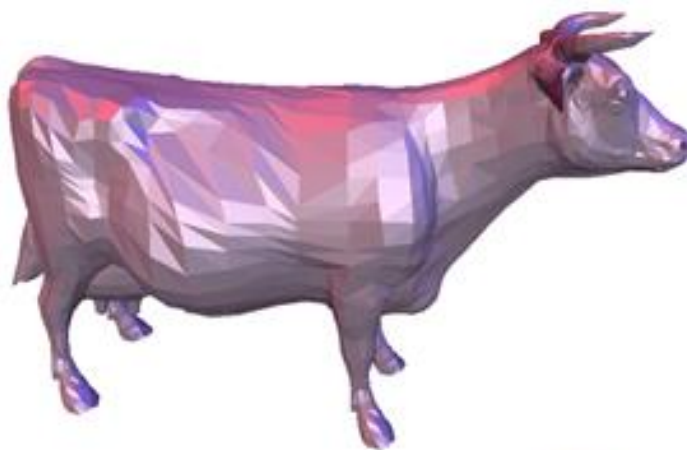
# Phong 明暗处理

- 由 Bui Tuong Phong 于 1973 年在他的博士论文中提出，与 Gouraud 明暗处理是不同的概念
  - Gouraud 明暗处理：对颜色插值
    - 计算量较小
    - 高光区域过渡不平滑
  - Phong 明暗处理：对法向插值
    - 计算量大于 Gouraud 明暗处理
    - 效果更加平滑
    - 现代图形硬件采用的明暗处理方法基本都是 Phong 明暗处理方法的变种



# 结果示例

Flat Shading



Phong Shading





Phong明暗处理能表示高光，是因为用了哪种插值？

- ☐ A 颜色插值
- ☐ B 重心坐标插值
- ☐ C 双线性插值
- ☒ D 法向插值

提交



# 本节课程内容

- 图形学中的一些重要概念
  - 颜色视觉
  - 图像和像素
  - 三角网格模型
  - Phong光照模型与明暗处理
  - 视点变换和视点方向
- 课程考核



# 视点变换和视点方向

- 图形学关注如何将由几何模型组成的三维场景绘制成高质量的彩色图像
- 变换在图形学中至关重要：通过变换，可以简洁高效地设置和编辑三维场景、光照位置和视点方向



## 为什么需要变换？

- 假设我们已经有了了一段可以绘制正方形  $[0,1]*[0,1]$  的代码：
  - **`drawUnitSquare(0,0,1,1);`**
- 现在我们需要绘制一个平行于坐标轴，且左下和右上顶点坐标分别为  $(lox, loy)$  和  $(hix, hiy)$  的矩形，我们应该怎么做？



# 一种解法

- 一种方法是写一段新的代码：

```
drawRect(lox, loy, hix, hiy) {  
    glBegin(GL_QUADS);  
    glVertex2f(lox, loy);  
    glVertex2f(hix, loy);  
    glVertex2f(hix, hiy);  
    glVertex2f(lox, hiy);  
    glEnd();  
}
```

- 矩形简单可以这么做，可是对于复杂的例子（例如绘制一个茶壶，大量的面片），怎么办？



# 利用变换的解法

```
drawRect(lox, loy, hix, hiy) {  
    glTranslate(lox, loy);  
    glScale(hix-lox, hiy-loy);  
    drawUnitSquare(0,0,1,1);  
}
```

- 这样的基于变换的解决方案在图形学中随时都需要用到；使用变换可以使代码更加快速、灵活、模块化
- 在光栅化渲染中，变换同样必不可少：
  - 通过MVP变换确定三维顶点坐标对应的二维像素坐标



# 什么叫变换 (Transformation)?

- 变换是一个将空间中的点  $x$  映射成其他点  $x'$  的函数
- 广泛应用于: Morphing, Deformation, Viewing, Projection, Real-time shadows...



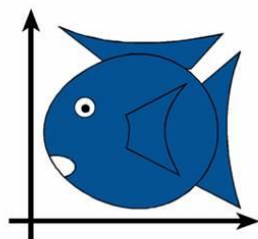
Fig 1. Undeformed Plastic

Fig 2. Deformed Plastic

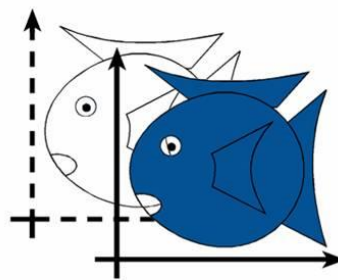


# 简单变换

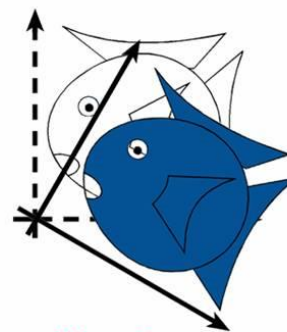
- 从左到右的变换分别是：
  - 不变 (Identity), 平移 (Translation), 旋转 (Rotation), 均衡缩放 (Isotropic scaling)
- 变换可以相互复合和嵌套：
  - 例如：先旋转，再缩放，最后再平移
- 简单变换都是可逆的



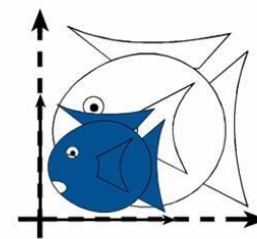
Identity



Translation



Rotation



Isotropic  
(Uniform)  
Scaling



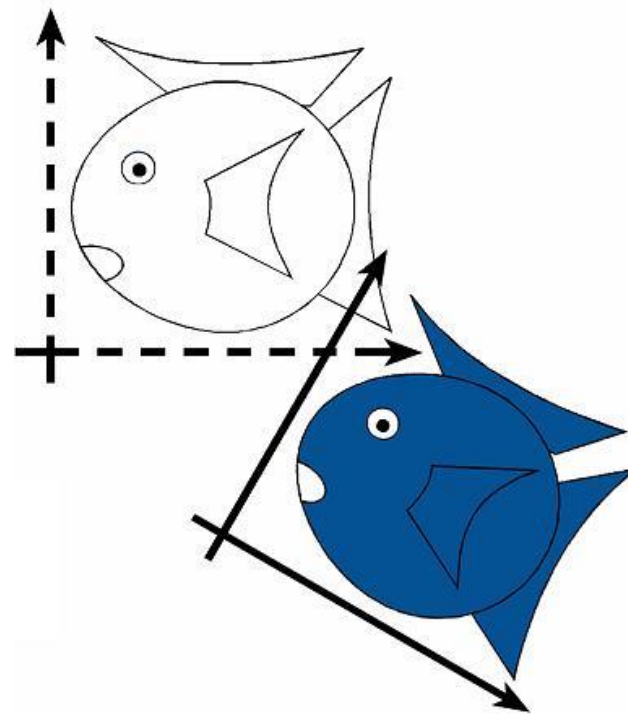
# 变换的分类

- 常见的变换有如下几类：
  - 刚体变换 (Rigid-body Transformation)
  - 相似变换 (Similarity Transformation)
  - 线性变换 (Linear Transformation)
  - 仿射变换 (Affine Transformation)
  - 投影变换 (Projective Transformation)



# 刚体变换

- 保持度量 (长度、角度、大小)。
- 刚体变换包括：
  - 不变
  - 平移
  - 旋转
  - 以及它们的复合

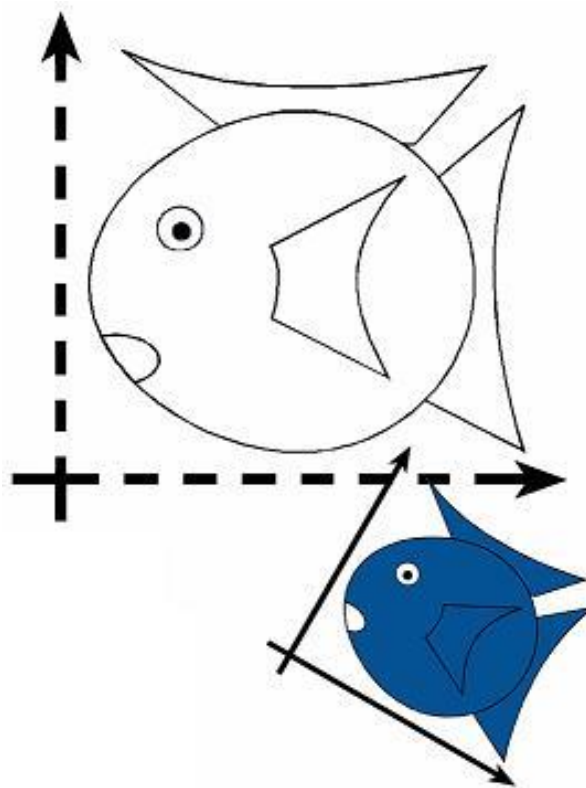


– 为何对称不是刚体变换？



# 相似变换

- 保持角度
- 相似变换包括：
  - 不变
  - 平移
  - 旋转
  - 均衡缩放
  - 以及它们的复合



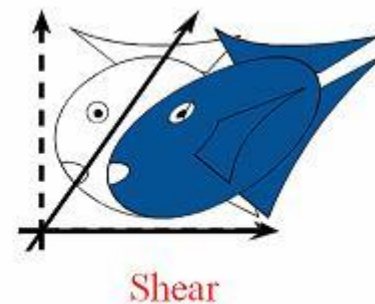
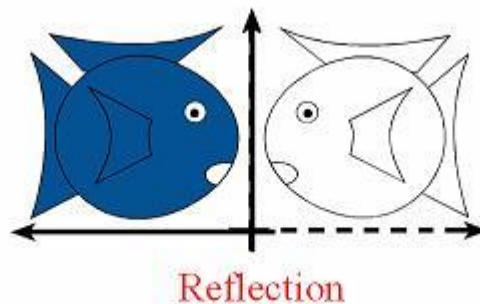
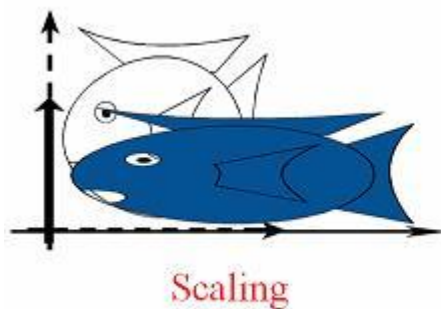


# 线性变换

- 线性变换满足如下方程：

$$L(p+q) = L(p) + L(q) \quad aL(p) = L(ap)$$

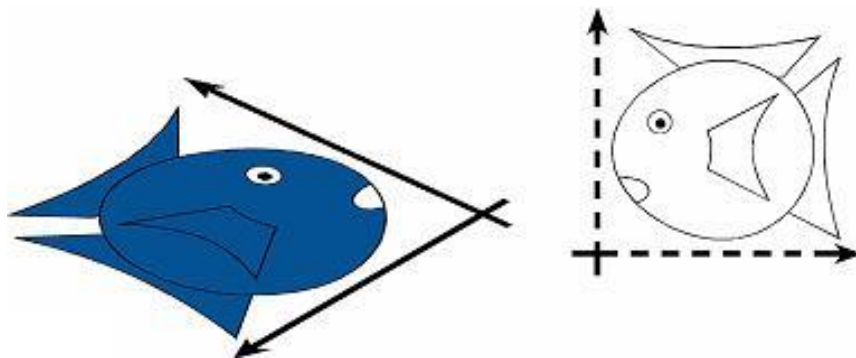
- 线性变换包括：
  - 不变、旋转、缩放 (不一定要均衡缩放)
  - 对称 (Reflection), 错切 (Shear)





# 仿射变换

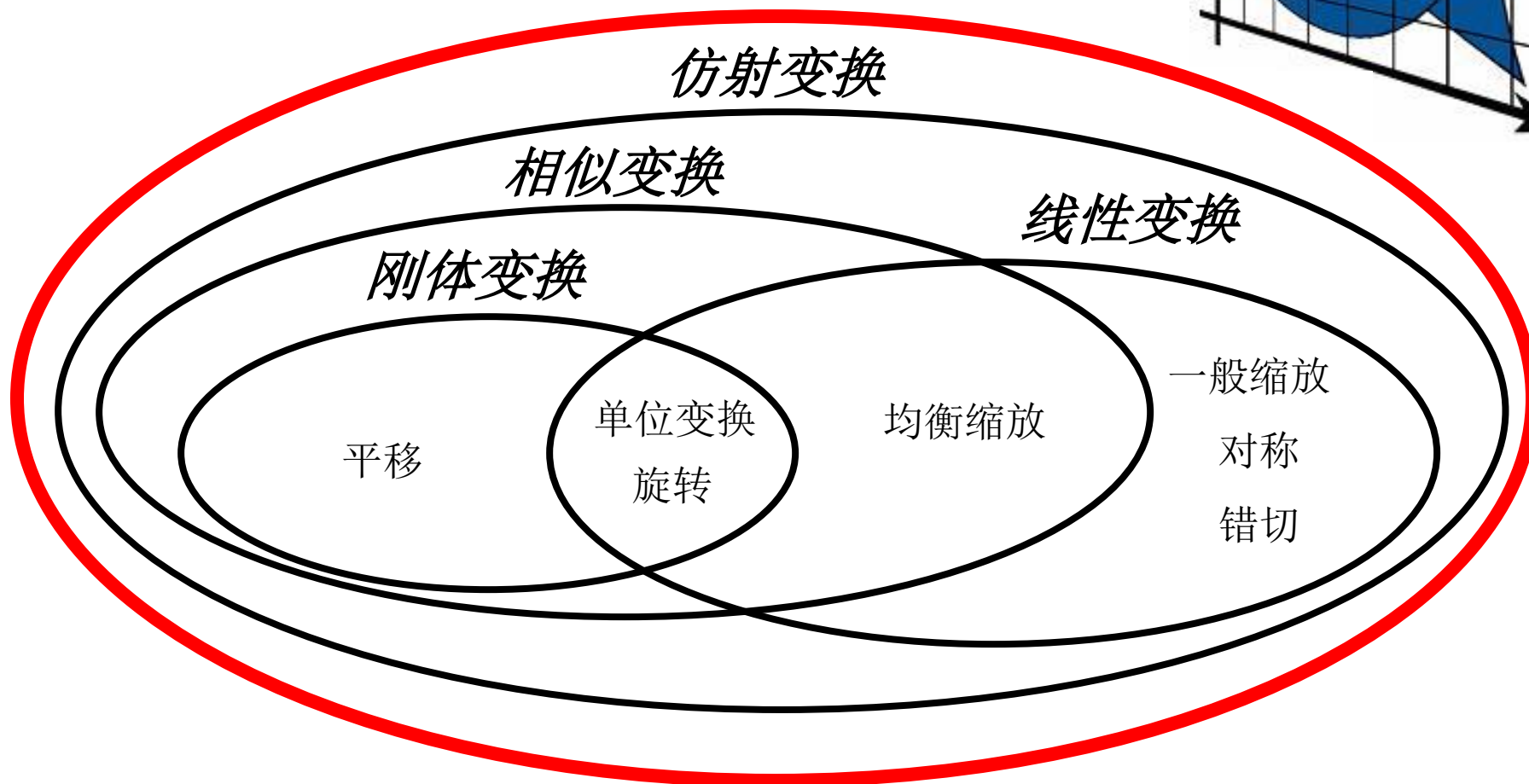
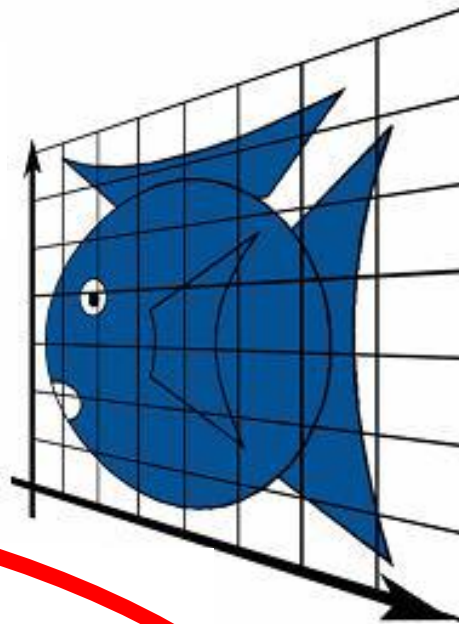
- 保持直线以及直线与直线平行关系
  - 两条在仿射变换之前平行的直线，在仿射变换之后依旧平行
- 仿射变换包括：
  - 线性变换
  - 相似变换
  - 以及它们的复合



# 投影变换

- 保持直线

投影变换





# 变换的表示

- 考虑简单的二维仿射变换:

$$x' = ax + by + c \quad (1)$$

$$y' = dx + ey + f \quad (2)$$

- 可以将 (1), (2) 统一地写成矩阵形式:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} a & b \\ d & e \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} c \\ f \end{bmatrix}$$

$$p' = M p + t$$



# 变换的表示

- 变换的表示涉及到了两个变量:  $M$  和  $t$

$$p' = M p + t$$

- 然而, 使用齐次坐标 (Homogenous Coordinates) 来表示  $p$  和  $p'$ , 则上述公式成为:

$$p' = M p$$

- 什么是齐次坐标?



# 齐次坐标 (Homogeneous Coordinates)

- 齐次坐标的本质是使用  $d + 1$  维数组来表示  $d$  维空间中的点和向量。

不使用齐次坐标表示

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} a & b \\ d & e \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} c \\ f \end{bmatrix}$$

$$p' = M p + t$$

使用齐次坐标表示

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$p' = M p$$



# 齐次坐标

- 由于引入了新的维度，在三维空间下，我们使用4X4的作用矩阵；同时，使用(x,y,z,w)来表示一个点或向量：

$$\begin{bmatrix} x' \\ y' \\ z' \\ w' \end{bmatrix} = \begin{bmatrix} a & b & c & d \\ e & f & g & h \\ i & j & k & l \\ m & n & o & p \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ w \end{bmatrix}$$

$$p' = \textcolor{red}{M} p$$



# 齐次坐标

- 在大部分情况下  $w = 1$ ，可以忽略。

$$\begin{bmatrix} x' \\ y' \\ z' \\ \color{red}{1} \end{bmatrix} = \begin{bmatrix} a & b & c & d \\ e & f & g & h \\ i & j & k & l \\ \color{red}{0} & \color{red}{0} & \color{red}{0} & \color{red}{1} \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ \color{red}{1} \end{bmatrix}$$

- 当齐次坐标被仿射矩阵作用时， $w$ 不会改变；而如果被投影矩阵作用， $w$ 会改变



# 齐次坐标的几何意义

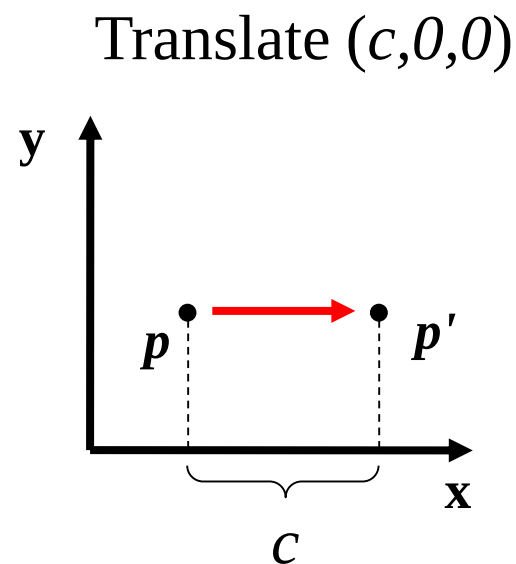
- 当  $w$  非零时，通过将所有四个坐标同时除以  $w$  以归一化
  - 例如： $(2x, 2y, 2z, 2)$  等价于  $(x, y, z, 1)$
  - 在投影变换中，这被称为[透视除法](#)
- 当  $w$  为零时，齐次坐标  $(x, y, z, 0)$  可以理解为沿  $(x, y, z)$  方向无穷远的点
  - 后面会看到， $w=0$ 的齐次坐标在平移矩阵作用下仍保持不变
  - 实际上描述的是向量（而不是点的坐标）



# 平移变换 ( $t_x, t_y, t_z$ )

- 可以看到，在齐次坐标的框架下，平移变换可以表示成为简单的矩阵乘法：

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

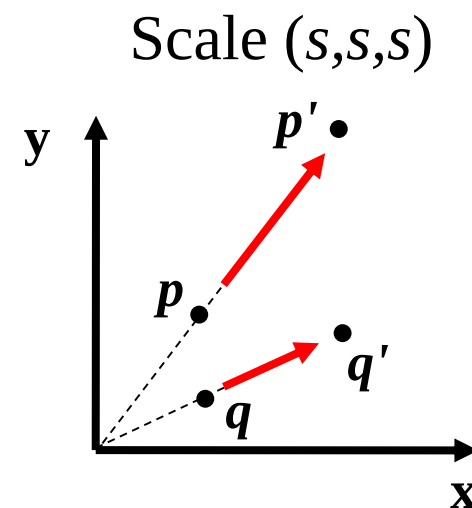




## 缩放变换 ( $s_x, s_y, s_z$ )

- 以原点为缩放中心的缩放变换可以表示成为：

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

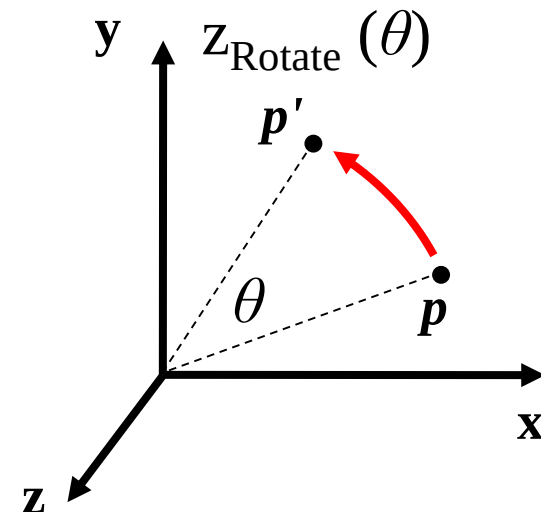




# 旋转变换

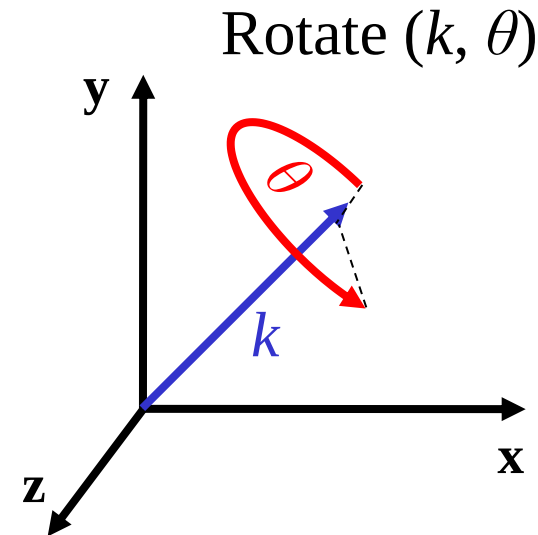
- 三维空间中物体的旋转具有三个独立的自由度
- 围绕  $z$  轴的旋转可以表示为：

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$



# 旋转变换

- 围绕单位向量  $(k_x, k_y, k_z)$  的旋转变换可以写成如下的Rodrigues 公式:



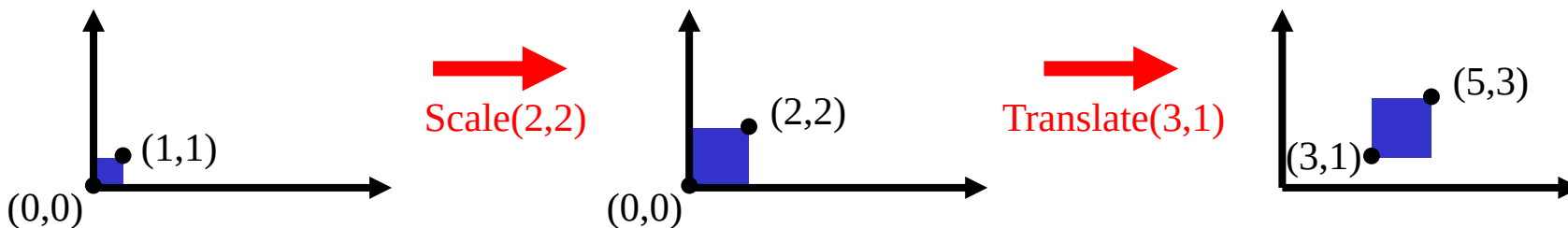
$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} k_x k_x (1-c) + c & k_y k_x (1-c) - k_z s & k_x k_z (1-c) + k_y s & 0 \\ k_y k_x (1-c) + k_z s & k_y k_y (1-c) + c & k_y k_z (1-c) - k_x s & 0 \\ k_z k_x (1-c) - k_y s & k_y k_z (1-c) + k_x s & k_z k_z (1-c) + c & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

where  $c = \cos\theta$  &  $s = \sin\theta$



# 变换的复合 (combination)

一个缩放和平移进行复合的例子：



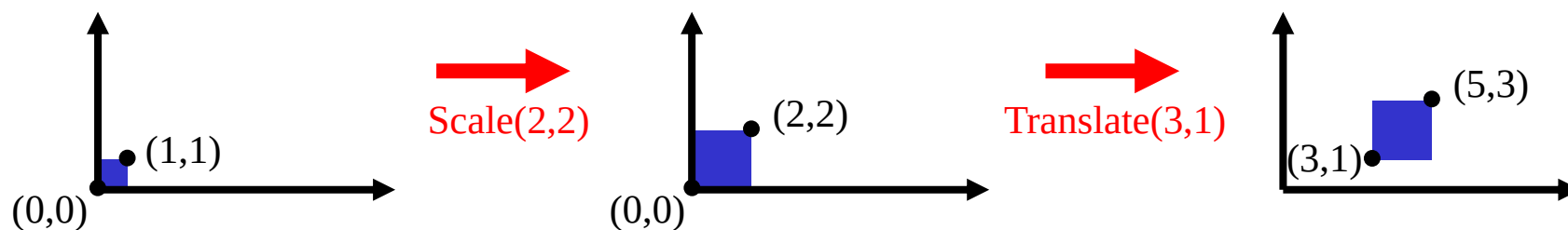
复合等价于矩阵乘法:  $p' = T(S p) = TS p$

$$TS = \begin{bmatrix} 1 & 0 & 3 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 2 & 0 & 3 \\ 0 & 2 & 1 \\ 0 & 0 & 1 \end{bmatrix}$$

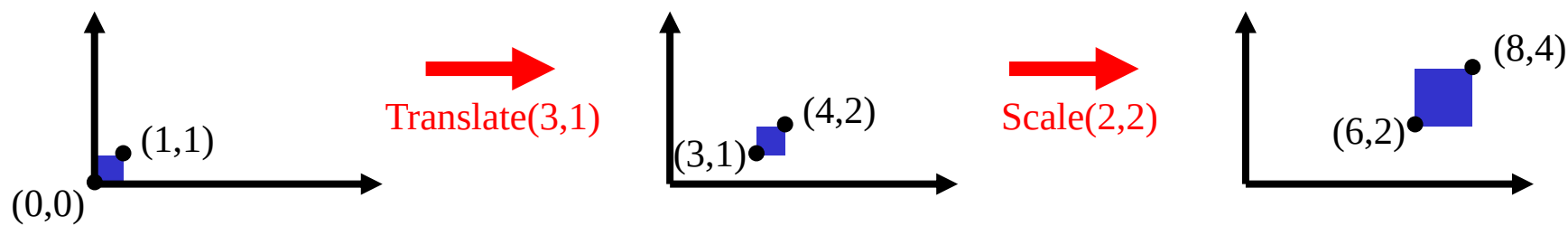


# 变换的复合不满足交换律

先缩放再平移:  $p' = T(S p) = TS p$



先平移再缩放:  $p' = S(T p) = ST p$





# 变换的复合不满足交换律

变换的复合不满足交换律，这是因为矩阵的乘法不满足交换律：

$$\mathbf{TS} \neq \mathbf{ST}$$

先缩放再平移：  $p' = T(S p) = TS p$

$$TS = \begin{bmatrix} 1 & 0 & 3 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 2 & 0 & 3 \\ 0 & 2 & 1 \\ 0 & 0 & 1 \end{bmatrix}$$

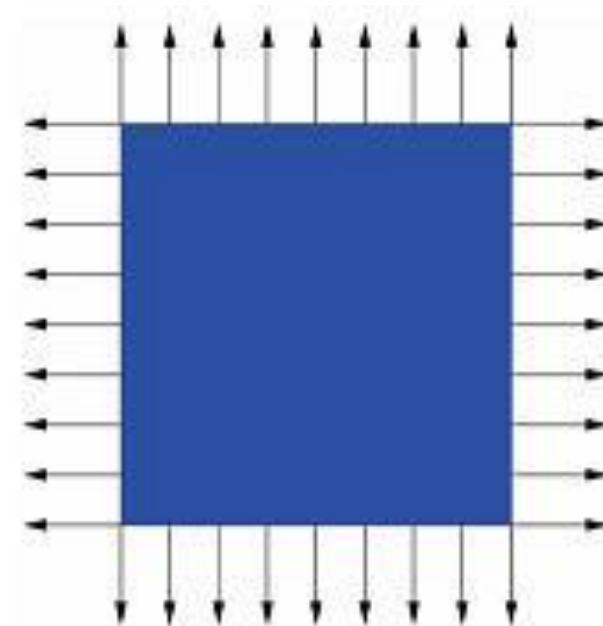
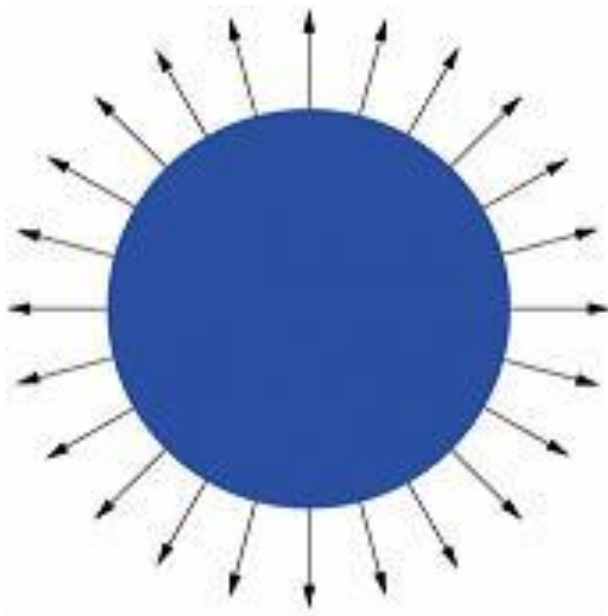
先平移再缩放：  $p' = S(T p) = ST p$

$$ST = \begin{bmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 3 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 2 & 0 & 6 \\ 0 & 2 & 2 \\ 0 & 0 & 1 \end{bmatrix}$$



# 法向量变换

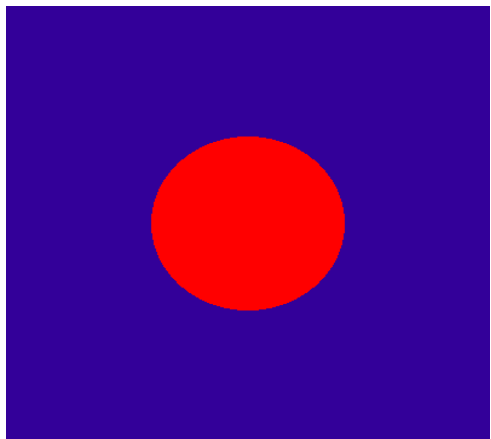
- 曲面的 (单位) 法向量是曲面与曲面正交的 (单位) 向量，它们是曲面最为重要的几何性质之一



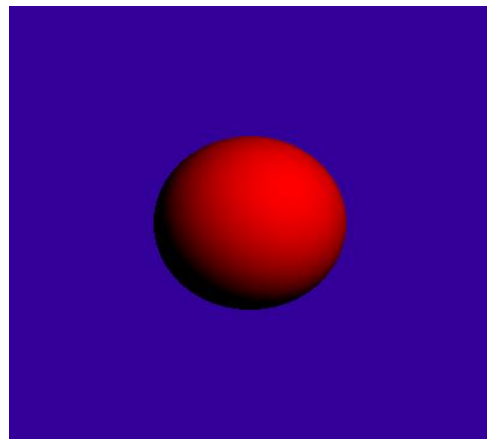


# 为什么法向量如此重要？

- 法向量是进行光照处理的必要输入，所有的光照模型都涉及到物体的法向量
- 只有知道了物体的法向量信息，才能绘制出具有三维立体感的图像



只有物体颜色

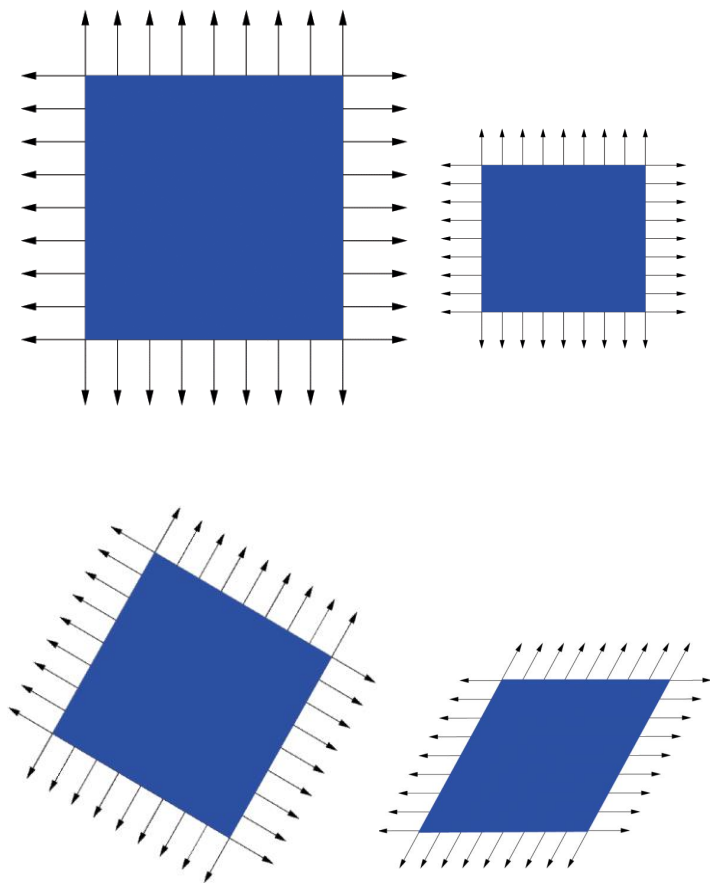


漫反射着色



# 像变换物体一样变换法向？

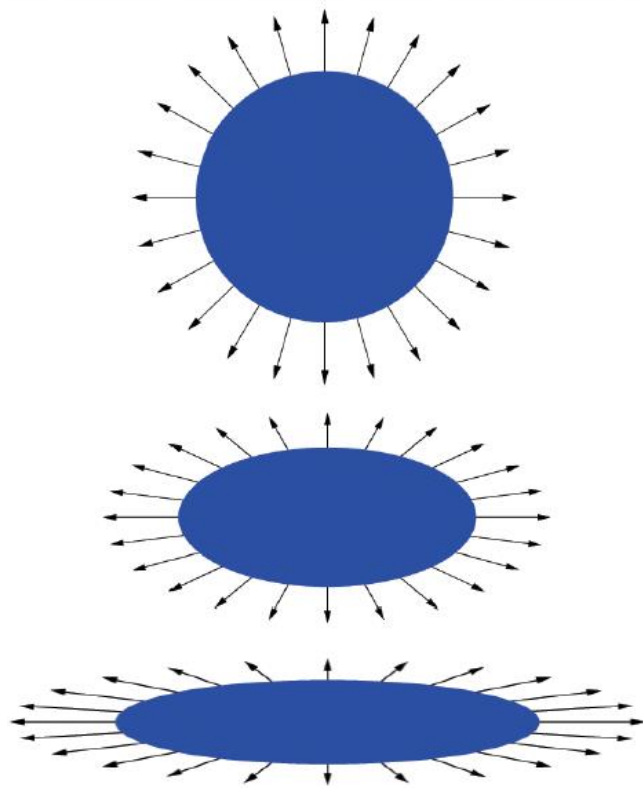
- 在我们对物体进行变换时，法向量可以类似变换吗？
- 我们发现，对于相似变换，法向量可以和物体**使用同样的变换方程**
- 然而对于右图中带有错切的仿射变换，**同样的方程则不适用**





# 像变换物体一样变换法向？

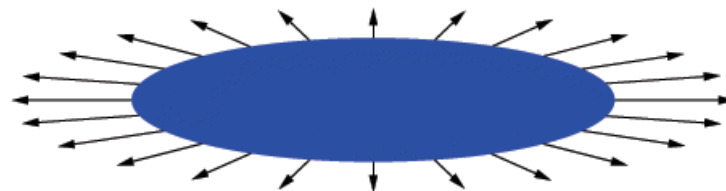
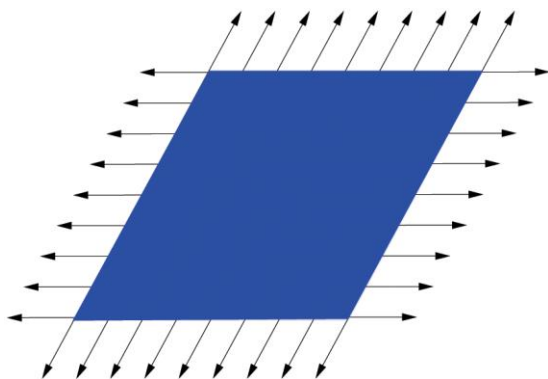
- 另一个例子（缩放）：
  - 对右图的球作非均衡的缩放，使用与物体同样的变换方程用于变换法向量会出现错误结果。



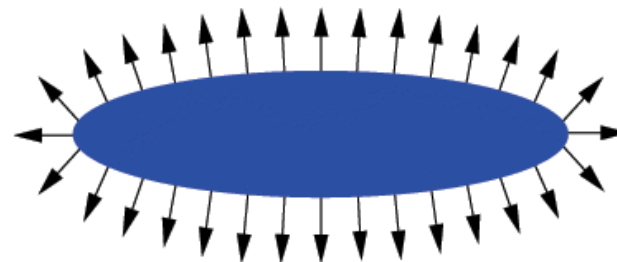
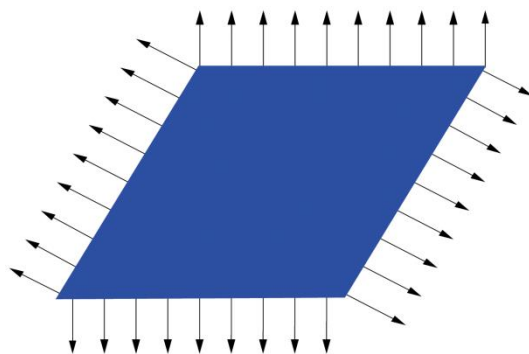


# 错切和缩放应有的法向变换

错误的法向变换



正确的法向变换

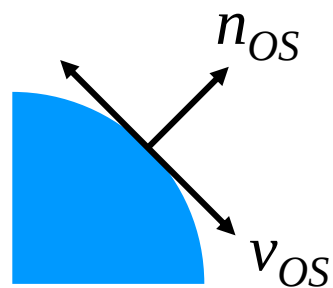


如何对法向量进行变换才能确保正确？

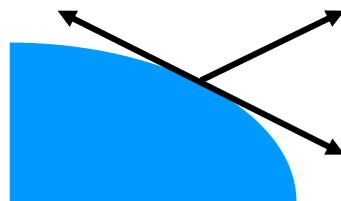


# 如何正确变换法向量?

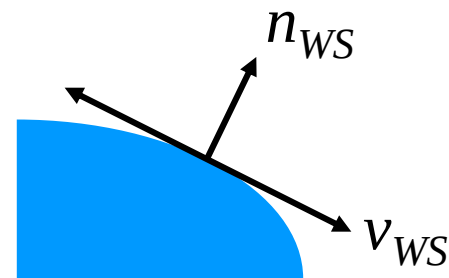
- 变换 **切平面 (Tangent Plane)**，再通过切平面计算法向量，而不是直接计算



原始表面



错误的变换



正确的变换

- 切平面上的任一向量  $v_{OS}$  变换后成为  $v_{WS}$  :

$$v_{WS} = \mathbf{M} v_{OS}$$



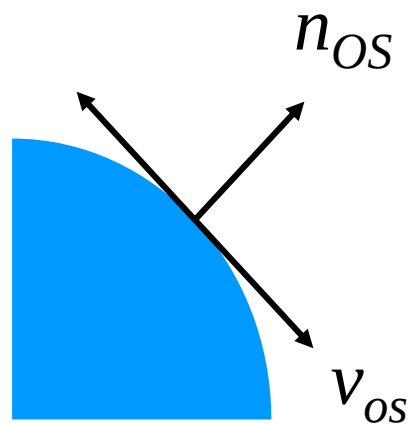
# 由切向量计算法向量

$\mathbf{v}_{OS}$  和  $\mathbf{n}_{OS}$  垂直:  $\mathbf{n}_{OS}^T \mathbf{v}_{OS} = 0$

$$\mathbf{n}_{OS}^T (\mathbf{M}^{-1} \mathbf{M}) \mathbf{v}_{OS} = 0$$

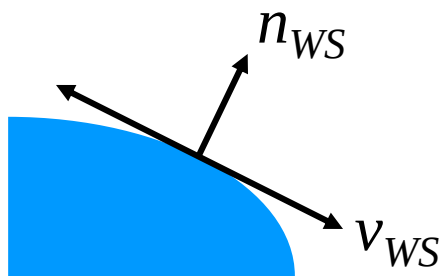
$$(\mathbf{n}_{OS}^T \mathbf{M}^{-1}) (\mathbf{M} \mathbf{v}_{OS}) = 0$$

$$(\mathbf{n}_{OS}^T \mathbf{M}^{-1}) \mathbf{v}_{WS} = 0$$



$\mathbf{v}_{WS}$  和  $\mathbf{n}_{WS}$  垂直:

$$\mathbf{n}_{WS}^T = \mathbf{n}_{OS}^T (\mathbf{M}^{-1})$$



$$\mathbf{n}_{WS} = (\mathbf{M}^{-1})^T \mathbf{n}_{OS}$$

法向量的变换矩阵是原变换矩阵的逆的转置



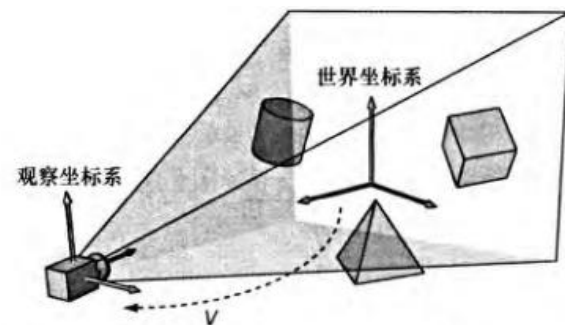
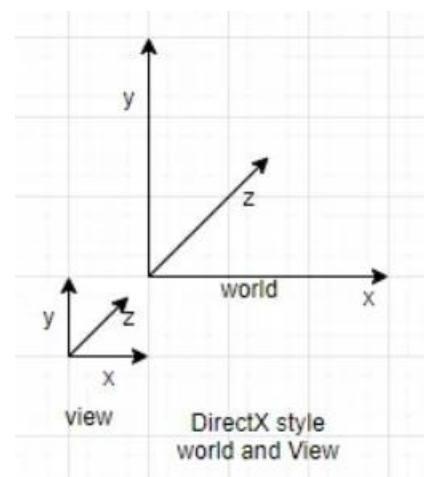
# 光栅化渲染流水线中的变换

- 如何确定待着色像素具体对应场景中哪个着色点？
  - 输入：三角网格（顶点&面）
  - 我们需要确定顶点的三维空间坐标与像素的二维屏幕坐标的映射关系！
  - 怎么确定？变换！
- 三个步骤：MVP变换
  - 世界变换/模型变换（World / Model Transform）
  - 视图变换（View Transform）
  - 投影变换（Projection Transform）



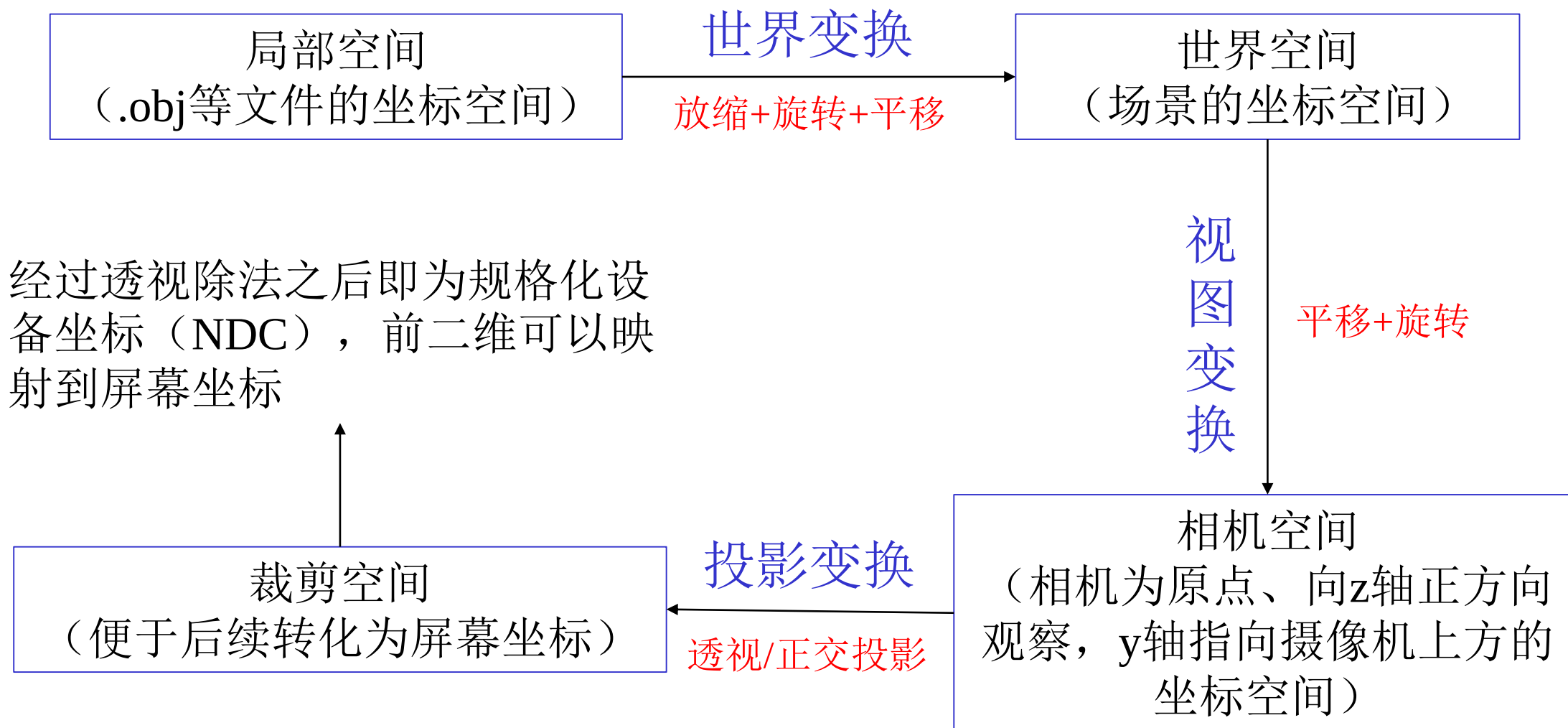
# MVP变换

- 三个步骤：MVP变换
  - 世界变换/模型变换（World / Model Transform）
    - 将物体从局部空间变换到世界空间
  - 视图变换（View Transform）
    - 将物体从世界空间变换到相机空间
    - 相机空间：（DirectX标准）左手系，摄像机位于原点，向z轴正方向观察，y轴指向摄像机上方
  - 投影变换（Projection Transform）
    - 将物体从相机空间变换到裁剪空间
    - 变换后经过透视除法（把w归一化），前二维可以和屏幕坐标建立映射





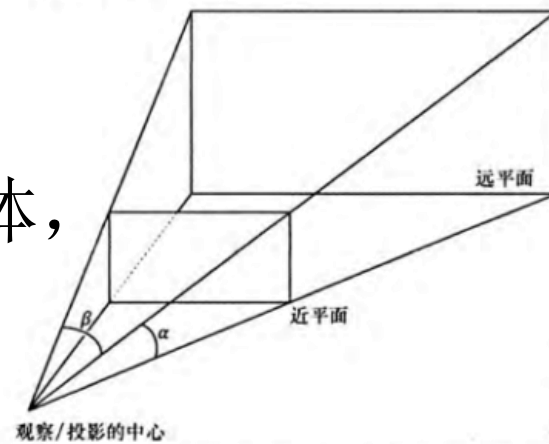
# 渲染流水线中的坐标空间





# 投影变换

- 我们的眼睛能将三维场景感知为二维图像，大脑则会将二维图像再重构回三维
  - 图形学中，用投影变换模拟
- 视锥：描述相机能看见的范围。实际上是一个截头体，即右图中远/近平面之间的几何体
- 投影变换的目的：
  - 把视锥（view frustum）变换为标准立方体（NDC坐标），从而获取对应屏幕坐标
  - NDC中，前三维坐标范围均为 $[-1,1]$
- 常用的包括正交投影和透视投影





# 投影变换

- （透视）投影变换：
  - 应用于四维齐次坐标  $(x, y, z, 1)$
  - 得到裁剪空间坐标  $(x', y', z', w)$
- 进行透视除法（除以 $w$ ）：
  - 得到标准化设备坐标NDC  $(x'/w, y'/w, z'/w, 1)$
  - 前三维坐标范围均为 $[-1, 1]$
- NDC前二维可以映射到屏幕的像素坐标
  - 便于后续进行深度测试、光栅化等操作



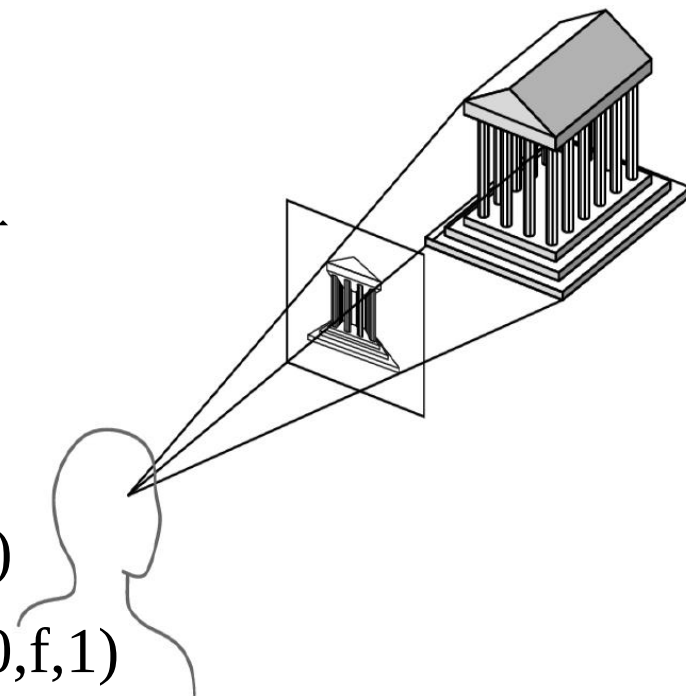
# 正交投影 (Orthographic Projection)

- 正交投影是视点在无穷远处的投影模式
- 缺乏立体透视效果
- 不需要进行透视除法
- 当  $xy$  平面是投影平面时，正交投影把  $(x, y, z)$  映射成为  $(x, y, 0)$



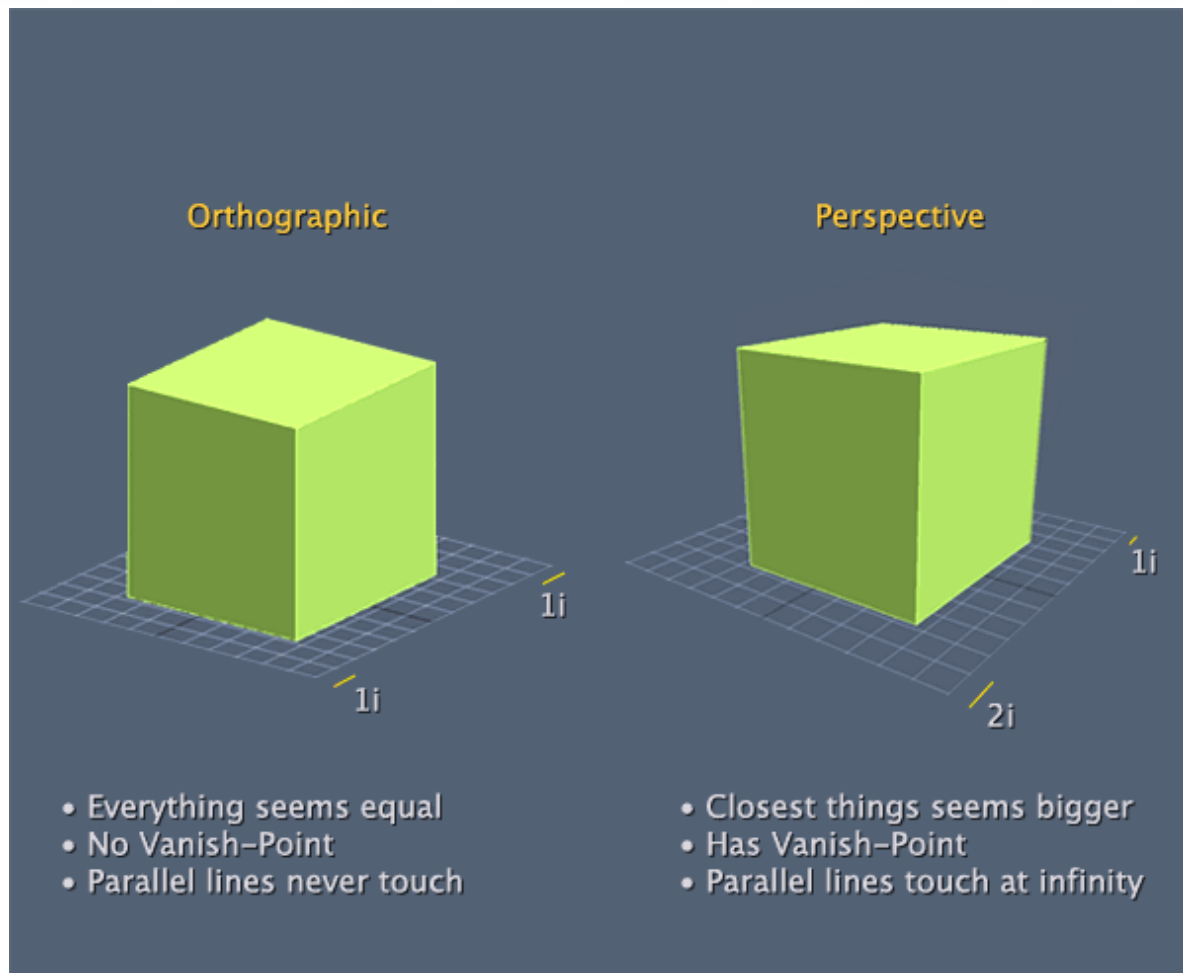
# 透视投影 (Perspective Projection)

- 透视投影是视点在有限距离处的投影模式
- 具有立体透视效果 – 近大远小
- 我们考虑简单情况:
  - 投影平面（近平面）为 $z = n$ ，远平面为 $z = f$ 时
  - 对于 $(x, y, z)$ 
    - 前两维映射成为  $((n/z)x, (n/z)y)$
    - 如果在近平面上，那么不变:  $(x, y, n, 1) \rightarrow (x, y, n, 1)$
    - 如果在远平面的中心，那么不变:  $(0, 0, f, 1) \rightarrow (0, 0, f, 1)$
  - 这对应一个矩阵变换





# 两种投影方式的对比





# 透视投影矩阵

- 使用齐次坐标描述的透视投影变换可以写成矩阵形式：

$$\begin{bmatrix} nx \\ ny \\ (n+f)z - nf \\ z \end{bmatrix} = \begin{bmatrix} n & 0 & 0 & 0 \\ 0 & n & 0 & 0 \\ 0 & 0 & n+f & -nf \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

- 已经把视锥变换为长方体；要变换为单位立方体，则再复合对应的缩放变换矩阵
- 最后进行透视除法，即可得到NDC



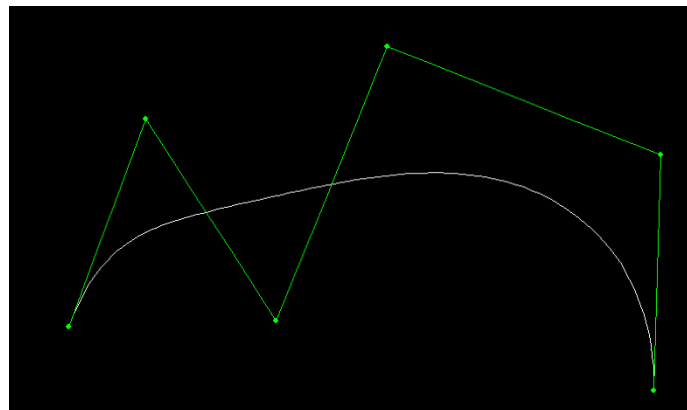
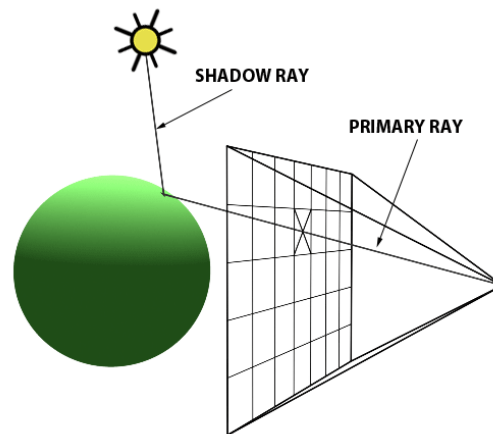
# 本节课程内容

- 图形学中的一些重要概念
  - 颜色视觉
  - 图像和像素
  - 三角网格模型
  - 光照模型与明暗处理
  - 视点变换和视点方向
- 课程考核



# 课程考核

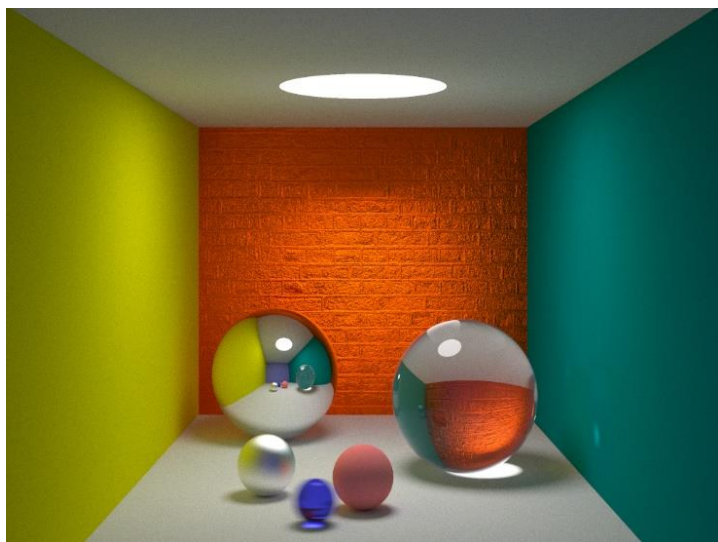
- 四个小作业
  - 光栅图形学
  - 光线投射
  - 样条曲线曲面
  - **Conditional GAN**
- 一个大作业，二选一
  - 真实感绘制（渲染）
  - 图形学算法挑战赛



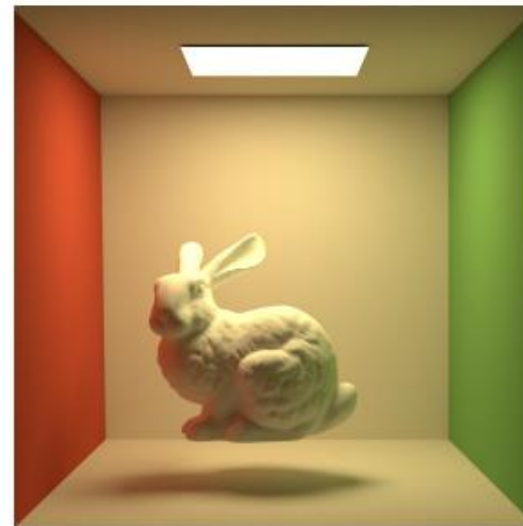


# 大作业-真实感绘制（渲染）

- 大作业
  - 支持NEE的路径跟踪绘制
    - 增加大作业文档
  - 报告+源码提交:



(b) 错误 MIS，没有正确计算重要性权重，着色错误



(c) 正确 MIS，和 NEE 最终结果一致，说明计算正确



# 往年优秀作业

- 计20，项思陶 (USC)





# 往年优秀作业

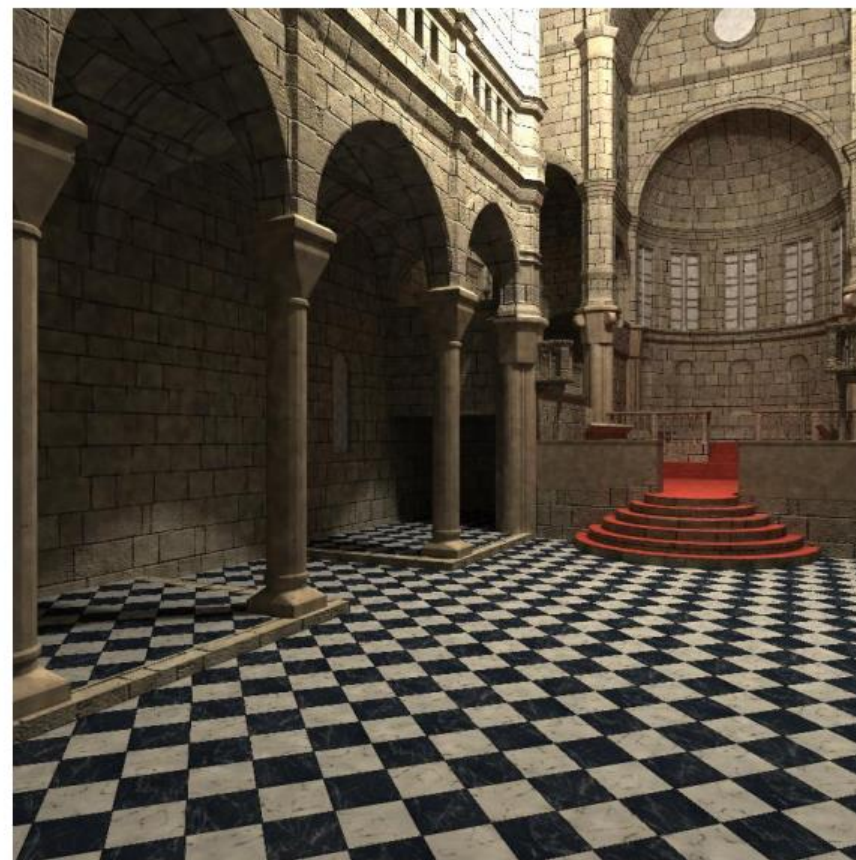
- 计30, 胡渊鸣 (MIT) 与冰雪奇缘





# 往年优秀作业

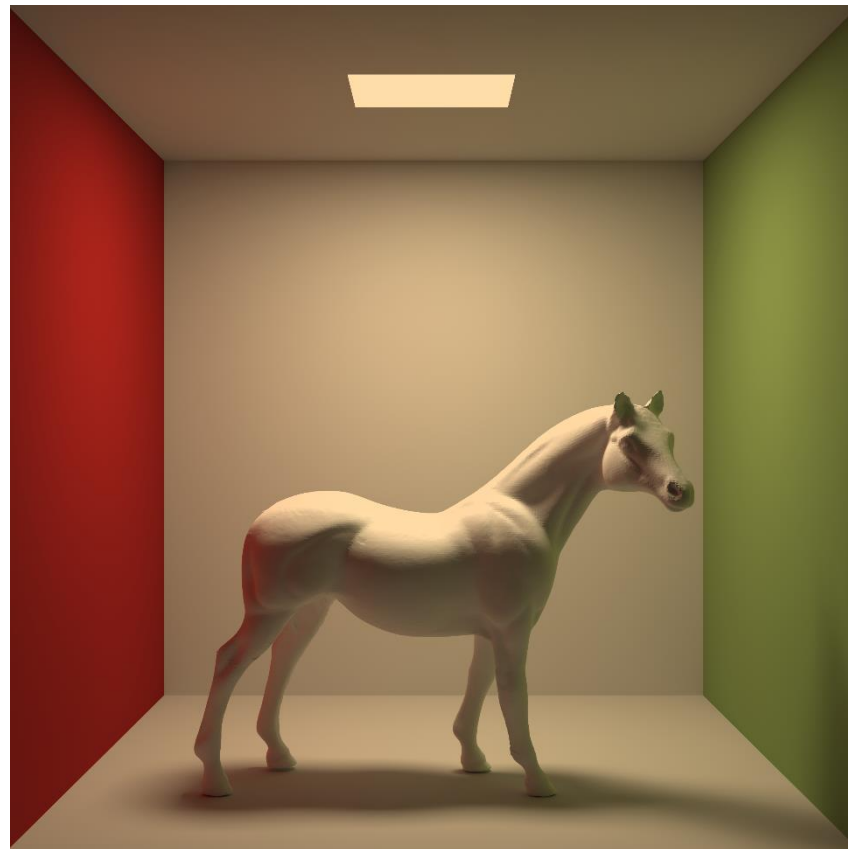
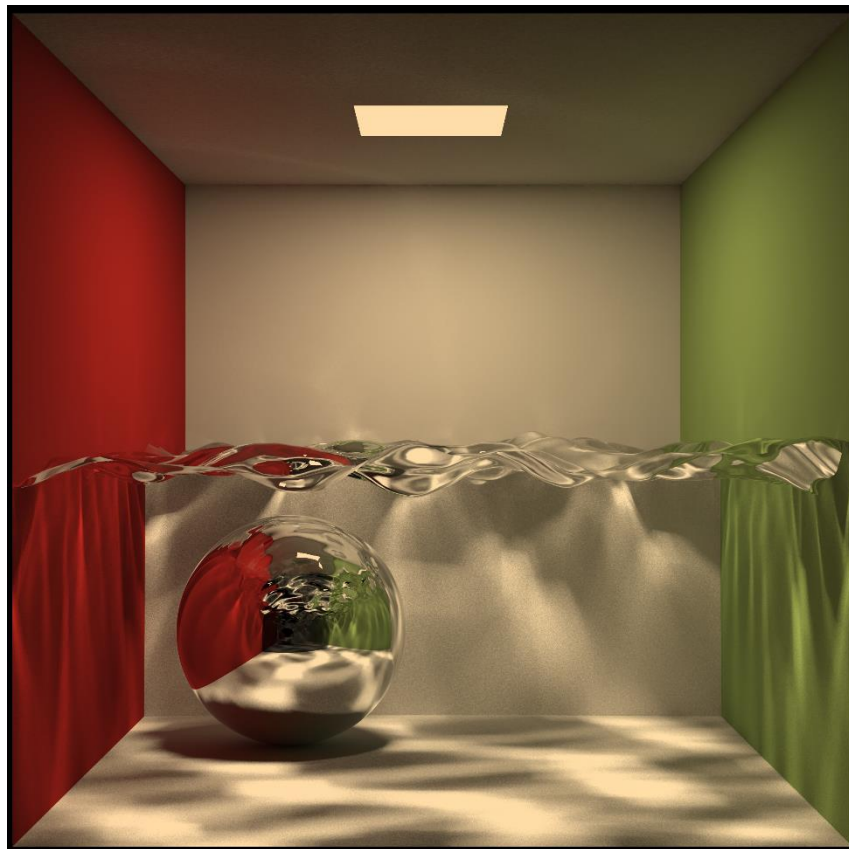
- 计33，古裔正





# 往年优秀作业

- 计30，王若松



# 2017年CS优秀作业



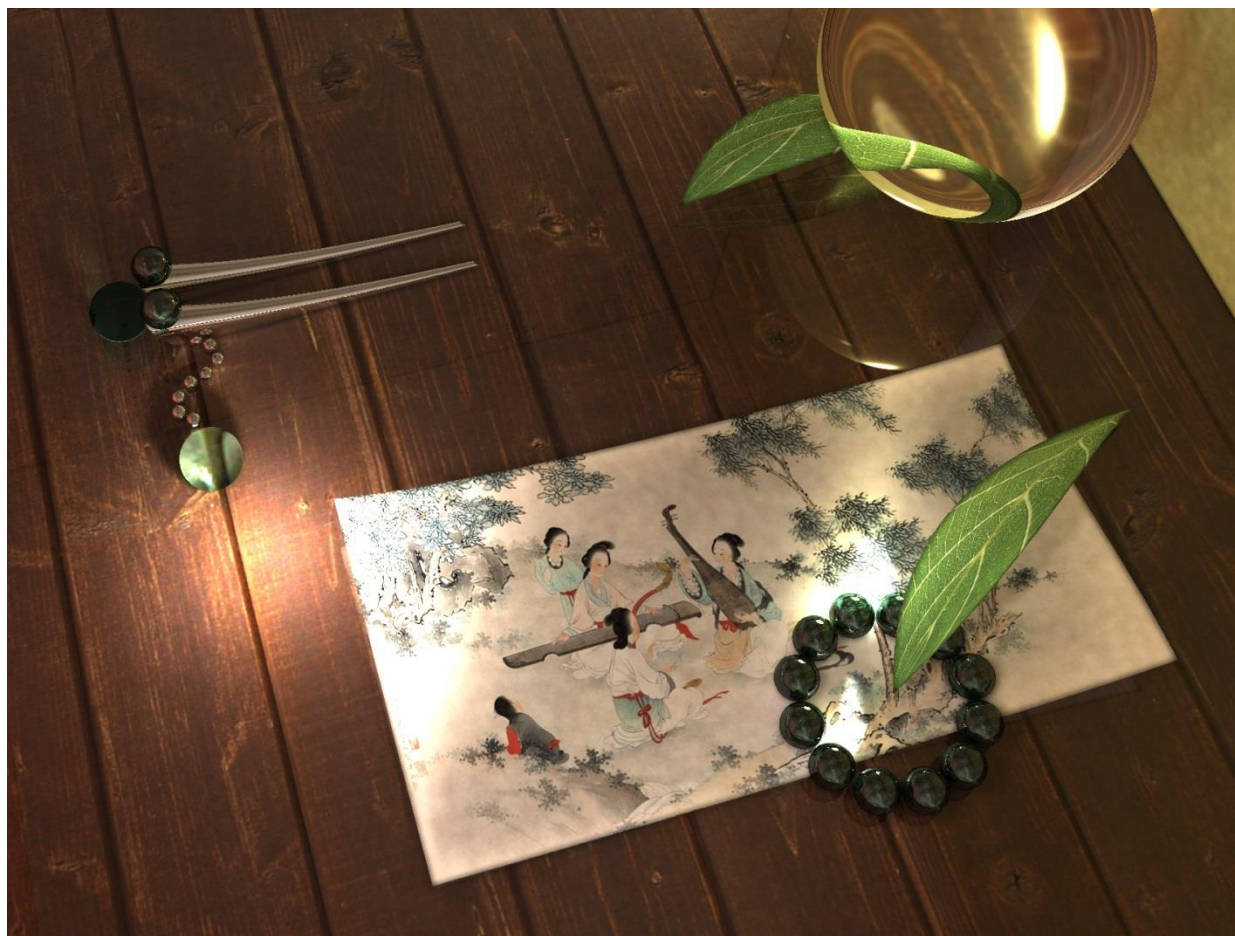
- 100分（秦岳）



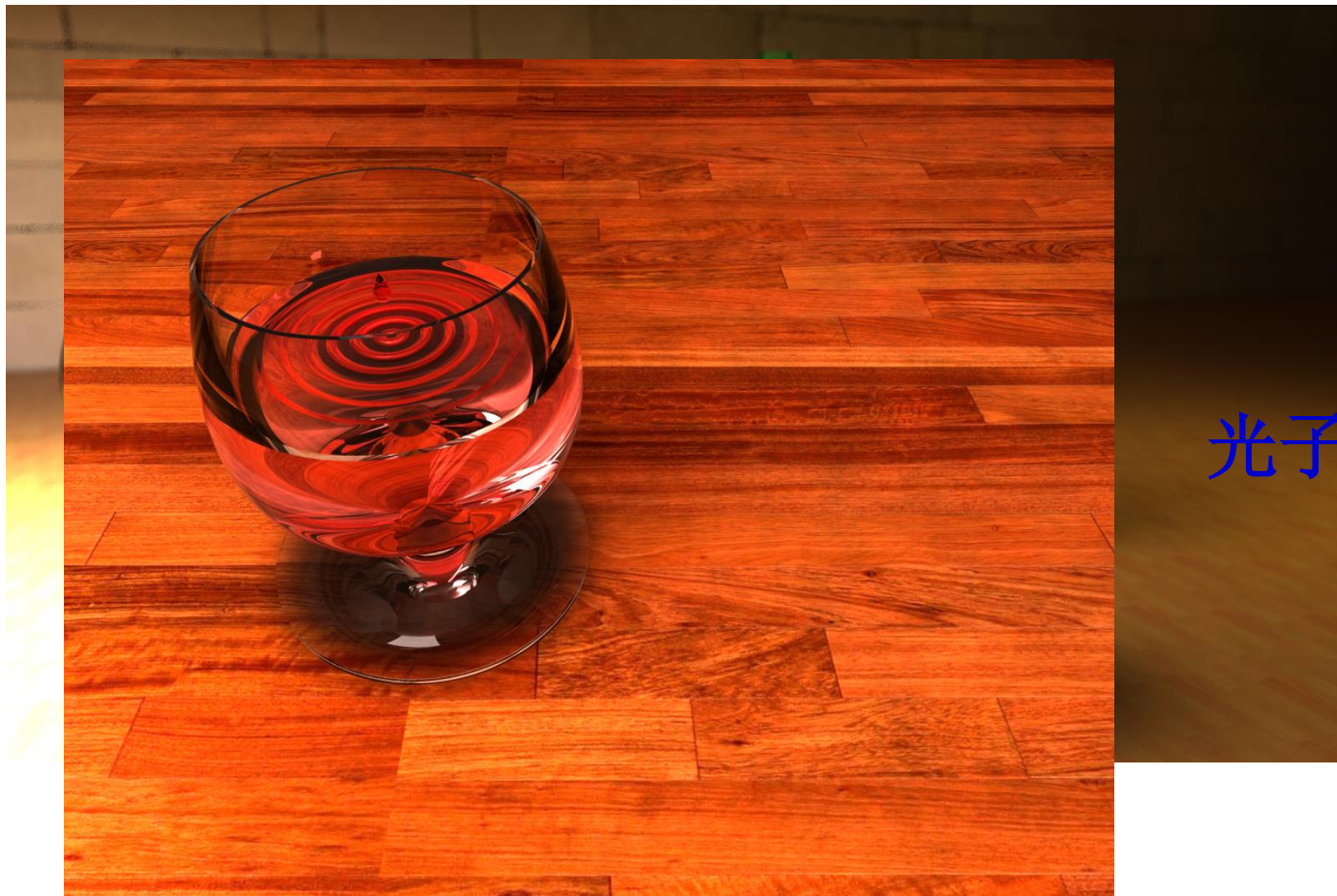


# 2017年CS优秀作业

- 97.5分



# 2017年Yao优秀作业



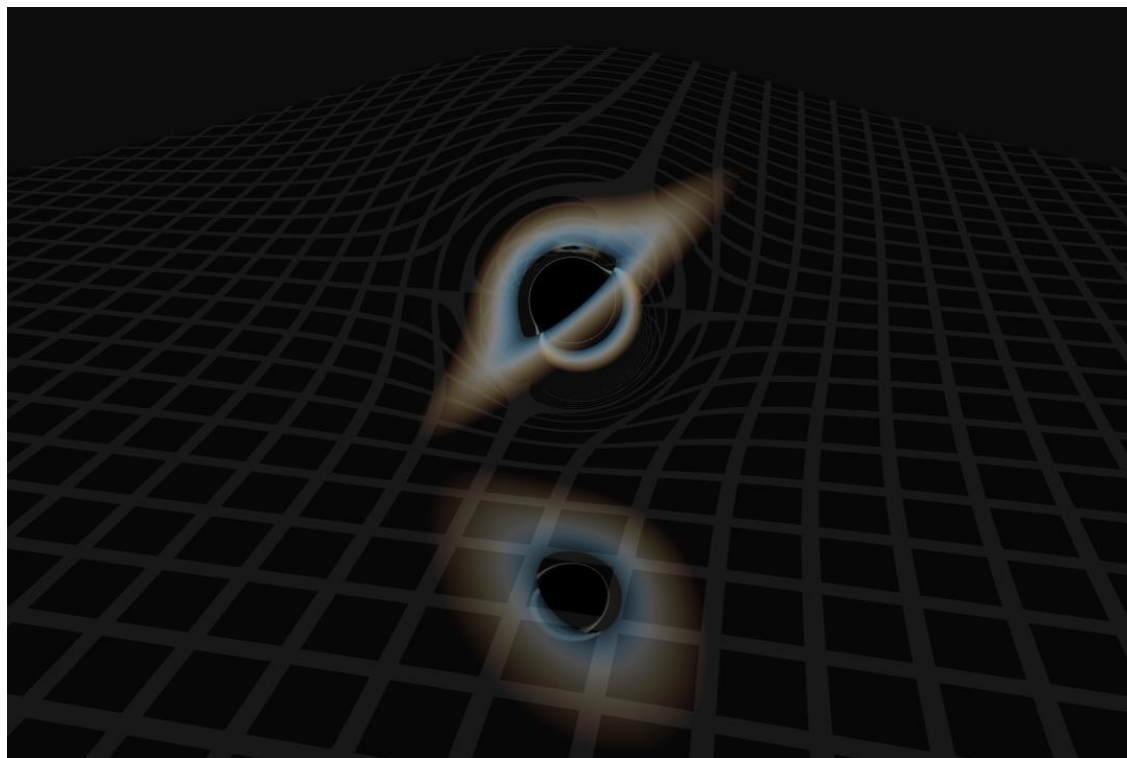
带景深

光子映射



# 2021年Yao优秀作业

- 刘方轩



弯曲时空下的  
光线追踪-黑洞



# 2024年优秀作业

- 濮成风：双向路径追踪





# 大作业-图形学算法挑战赛

- 25年计图挑战赛具体安排将在3月发布。
- 24年计图挑战赛：奖金总额28万；荣誉证书；腾讯校招绿色通道。
- 每个队伍2-3人。

# 24年优秀队伍成果



qwqw队:

参考图



生成图 1



desk

生成图 2



television

生成图 3



cabinet



clock



swan



owl



# 24年优秀队伍成果



pad pad eos队:

参考图

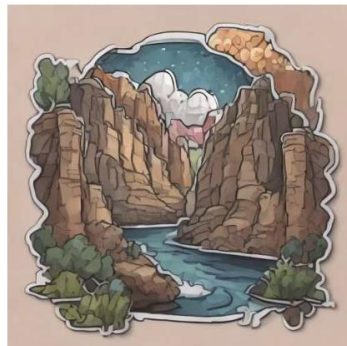


生成图1



Burger

生成图2



Canyon

生成图3



Swan



Burger



Dolphin



Train

# 24年优秀队伍成果



pad pad eos队:

参考图

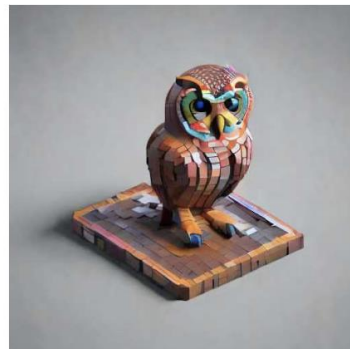


生成图1



Grass

生成图2



Owl

生成图3



Soda



Zebra



Bus



Pants



# 今日人物: Pat Hanrahan

- Pat Hanrahan, Stanford 教授
  - Google scholar 的H因子98
  - 论文300+篇, 引用58226次
    - Light field 6410, Volume rendering 2943
  - University of Wisconsin–Madison PHD(1985)
  - 纽约工学院、Pixar
  - 1989年, 受聘Princeton, 1995年受聘Stanford
  - 1993获SIGGRAPH成就奖, 2003年获Coons奖
  - 2004年获奥斯卡奖, 2020年图灵奖。
  - 美国工程院, 艺术与科学院 院士, ACM Fellow





谢谢！